
Código de Olas: El Misterio del Puerto de Ancón

Una Historia de Surf, Tecnología y el Secreto del Mar

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

Código de Olas: El Misterio del Puerto de Ancón

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados.

Autor: Alex Goyzueta Delgado

Edición: Alex Goyzueta Delgado

Agradecimientos:

A Ancón, y sus surfistas y soñadores.

A la comunidad Python del Perú.

A los guardianes del mar.

Datos de catalogación:

Goyzueta Delgado, Alex

Código de Olas: El Misterio del Puerto de Ancón / Alex Goyzueta Delgado

1ra edición — Lima, 2026

ISBN: 978-612-XX-XXXX-X

Dedicatoria

A Ancón.

A su mar eterno.

A sus olas que guardan secretos.

A sus surfistas que cabalgan sueños.

A los que creen que el código y el océano
hablan el mismo lenguaje:

el de la persistencia.

"El mar no da tregua,

pero tampoco olvida."

— Proverbio de surfistas de Ancón

Prefacio

Ancón existe

Si alguna vez has estado en Ancón, sabes de qué hablo. El olor a salitre. El sonido de las olas rompiendo en el muelle. La brisa que trae historias de navegantes y pescadores.

Si no has estado, no importa. Este libro te llevará allí.

¿Por qué este libro?

Escribí **Código de Olas** porque crecí en Ancón viendo el mar todos los días. Mi padre y mis abuelos fueron anconeros, y el puerto corre por mis venas. Este libro une dos mundos que amo: la tecnología y la costa que me vio crecer.

El código, como el mar, requiere paciencia y observación. Y cuando entiendes sus patrones, puedes descifrar cualquier misterio.

¿Cómo usar este libro?

Cada capítulo tiene tres partes:

1. **Narrativa** — La historia de Mateo, un analista de datos y surfista de Ancón que descubre un misterio en el mar.
2. **Código** — Los conceptos de Python que Mateo usa para resolver los enigmas del océano.
3. **Enigmas** — Ejercicios para que tú descifres las olas digitales.

¿Qué necesitas?

- Python 3.10+ instalado
- Curiosidad
- Ganas de aprender

Al terminar, no solo sabrás quién está contaminando las olas de Ancón. Sabrás Python desde cero hasta programación orientada a objetos. Y quizás, solo quizás, aprendas a leer el lenguaje del mar como yo aprendí a leerlo desde niño.

— *Alex Goyzueta Delgado*

Ancón, Lima — 2026

Introducción

Ancón, 2026

Ancón no es cualquier playa. Es la playa de Lima que vio nacer a generaciones de surfistas. Su muelle, el más largo de la costa peruana, se adentra en el Pacífico como un abrazo de piedra. Sus olas, consistentes y poderosas, han formado a campeones nacionales.

Pero Ancón guarda secretos.

Bajo sus aguas, hay restos de naufragios coloniales. En su cementerio histórico, descansan personajes de la República. Y en sus acantilados, hay cuevas que pocos conocen.

Y ahora, alguien está usando tecnología submarina para manipular las olas.

El Misterio

Las olas de Ancón están cambiando. No de forma natural. Los surfistas locales notaron que ciertos patrones se repiten con exactitud matemática. Como si alguien estuviera programando el mar.

Mateo Sánchez, surfista y programador autodidacta, descubre que los patrones de las olas son código: **Olas Digitales**, datos cifrados en la frecuencia y altura del agua.

Alguien está enviando mensajes a través del mar. Y esos mensajes revelan una conspiración para privatizar las playas del Perú.

¿Qué son las Olas Digitales?

Si los quipus digitales eran nudos en cuerdas que codificaban información, las **Olas Digitales** son patrones en el agua que codifican datos. La frecuencia de la ola es un bit. La altura es otro. La dirección es un tercero.

Tres variables. Como tres tipos de datos en Python.

Y Mateo es el único que puede descifrarlas.

El conflicto

Una corporación internacional quiere construir un complejo turístico privado en Ancón. Para ello, necesita demostrar que las olas son "peligrosas". Están usando un generador de olas submarino para crear patrones erráticos y asustar a los surfistas.

Pero algo salió mal. El generador también está enviando mensajes cifrados. Mensajes que Mateo está a punto de descifrar.

Y hay personas dispuestas a todo para que esos mensajes no sean revelados.

Capítulo 1: El Secreto de la Playa

Conceptos: Variables, tipos de datos, ``print()``, f-strings

Mateo Sánchez remó contra la corriente. Eran las 6:15 a.m. y el sol apenas comenzaba a teñir de naranja el horizonte del Pacífico. Desde su tabla, Ancón se veía distinto: el muelle de piedra alargándose como un brazo dormido, las caletas vacías, las casas de colores subiendo por el cerro.

Era su rutina. Madrugar, revisar el parte de olas en su teléfono, y meterse al agua antes de que el viento arruinara la superficie.

Pero hoy algo era diferente.

La ola que venía no se comportaba como debía. Rompió en un punto extraño, demasiado perfecto. Como si alguien hubiera trazado una línea recta en el agua.

—¿Viste eso? —le gritó su amigo Rafa, desde la orilla.

—Lo vi —respondió Mateo—. Parece... código.

—¿Código?

—Como cuando programo. Patrones. Esto no es natural.

Mateo guardó silencio. Desde pequeño había sido el raro: surfista y programador. Mientras sus amigos veían YouTube, él aprendía Python en una laptop prestada de la biblioteca municipal de Ancón. Mientras ellos salían de fiesta, él se quedaba descifrando APIs.

Y ahora, el mar le estaba hablando en el único idioma que entendía: datos.

La primera ola digital

Mateo salió del agua, tomó su laptop y comenzó a escribir. Lo primero: entender qué datos tenía.

En Python, todo comienza con **variables**. Son como cajas donde guardas información del mundo real. Esa información tiene un **tipo**: números, texto, booleanos.

```
# =====  
# OLA DIGITAL: ANÁLISIS DE PATRONES  
# Surfista: Mateo Sánchez  
# Fecha: 27 de junio, 2026  
# =====  
  
# --- DATOS DE LA OLA ---  
  
nombre_playa = "Ancón"  
hora_observacion = "06:15"
```

```

altura_ola = 1.8 # metros
frecuencia_ola = 12 # segundos entre olas
temperatura_agua = 18.5 # grados Celsius
direccion_viento = "sur-oeste"
es_anomalia = True

print("=== INFORME DE OLA ===")
print("Playa:", nombre_playa)
print("Hora:", hora_observacion)
print("Altura:", altura_ola, "metros")
print("Frecuencia:", frecuencia_ola, "segundos")
print("Anomalía detectada:", es_anomalia)

```

Pero la forma más elegante de mostrar datos en Python es con **f-strings**:

```

print(f"\n--- Resumen ---")
print(f"Playa {nombre_playa} a las {hora_observacion}")
print(f'Ola de {altura_ola}m cada {frecuencia_ola} segundos')
print(f'¿Anomalía? {es_anomalia}')

```

El primer patrón

Mateo observó los datos más de cerca. La ola no solo era extraña: tenía una firma. Como una variable en Python que guarda un valor específico, la ola guardaba información en su estructura.

—Si la ola fuera una variable —murmuró—, tendría un nombre, un tipo y un valor.

Anotó en su laptop:

```

# --- TIPOS DE DATOS DE LA OLA ---

nombre_ola = "Ancón_01"          # str - texto
altura_maxima = 2.4               # float - decimal
frecuencia_segundos = 12          # int - entero
es_generada_artificialmente = True # bool - booleano (True/False)

print(f"\nOla: {nombre_ola}")
print(f'Tipo de nombre_ola: {type(nombre_ola)}')
print(f'Tipo de altura_maxima: {type(altura_maxima)}')
print(f'Tipo de frecuencia_segundos: {type(frecuencia_segundos)}')
print(f'Tipo de es_generada: {type(es_generada_artificialmente)}')

```

El resultado le confirmó algo: la ola tenía datos medibles. Pero necesitaba más. Necesitaba el parte de olas completo.

El parte de olas digital

El amigo de Mateo, Rafa, trabajaba en la Capitanía del Puerto de Ancón. Tenía acceso a los datos históricos de oleaje. Llamó por teléfono:

—Rafa, ¿me puedes pasar los datos de las últimas 48 horas?

—¿Para qué?

—Hay algo raro en el agua. Las olas tienen patrones que se repiten. Como código binario.

—Estás loco.

—Puede ser. Pero pásame los datos.

Rafa le envió un archivo. Mateo lo abrió y comenzó a analizarlo:

```
# --- PARTE DE OLAS DE ANCÓN ---

fecha_parte = "27-06-2026"
total_olas_registradas = 147
ola_mas_alta = 2.8
ola_promedio = 1.6
direccion_olas = "SO"

print(f"\n=== PARTE DE OLAS: {fecha_parte} ===")
print(f"Total de olas: {total_olas_registradas}")
print(f'Ola más alta: {ola_mas_alta}m')
print(f'Promedio: {ola_promedio}m')
print(f'Dirección: {direccion_olas}')

# Porcentaje de olas anómalas
total_anomalias = 23
porcentaje = (total_anomalias / total_olas_registradas) * 100
print(f'Anomalías: {total_anomalias} ({porcentaje:.1f}%)')
```

—23 anomalías en 48 horas —dijo Mateo—. Eso es demasiado. En un mar natural, las anomalías son menos del 5%. Aquí estamos en 15.6%.

Alguien está generando olas artificiales. Y esas olas están codificando algo.

Tu primer enigma

Enigma 1.1: Tu parte de olas

Crea variables para los siguientes datos de una playa:

- `nombre\_playa`: "Ancón Norte"
- `temperatura\_agua`: 19.2
- `altura\_ola`: 2.1
- `es\_segura`: True
- `hora\_medicion`: "07:30"

Muéstralos con `print()` y f-strings.

Enigma 1.2: Calcula el promedio

Tienes 5 alturas de ola: 1.5, 2.0, 1.8, 2.2, 1.9. Calcula el promedio y muéstralo. Pista: suma las alturas y divide entre 5.

Enigma 1.3: La ola binaria

El patrón de la ola se codifica así: `1.8:0.5:2.1` (altura:frecuencia:velocidad). Separa los valores usando `.split(":")` y muestra cada uno por separado.

Lo que aprendiste

- Las **variables** guardan información en la memoria

- Los **tipos básicos** son: ``int``, ``float``, ``str``, ``bool``
- ``print()`` muestra datos en pantalla
 - Los **f-strings** (``f"texto {variable}"``) incrustan variables en texto
- ``type()`` revela el tipo de una variable
- ``split()`` separa texto en partes

Mateo cerró su laptop. Tenía datos. Tenía patrones. Pero lo más inquietante estaba por venir.

Cuando revisó el parte de olas más detenidamente, notó que las anomalías seguían un patrón horario: ocurrían exactamente cada 3 horas. Como si un programa estuviera ejecutando un bucle.

—Alguien está programando el mar —dijo en voz alta—. Y voy a descubrir quién.

Capítulo 2: El Lenguaje del Mar

Conceptos: Strings, métodos de string, slicing

Eran las 2:00 a.m. cuando Mateo se despertó sobresaltado. El sonido del mar golpeando el muelle de Ancón era distinto. Más rítmico. Casi musical.

Se puso los shorts y salió corriendo a la playa. La luna llena iluminaba el agua como un reflector plateado. Y entonces lo vio.

Las olas no solo tenían patrones: tenían **mensajes**.

Cada ola, al romper, dejaba una línea de espuma en la orilla. La espuma no se dispersaba al azar: formaba líneas rectas. Como código de barras en la arena.

—Son strings —susurró Mateo—. Cadenas de caracteres escritas en el agua.

Strings: El alfabeto del mar

Los **strings** son secuencias de caracteres. Como las olas: una tras otra, formando un mensaje.

Mateo corrió a su laptop y comenzó a transcribir los patrones de espuma:

```
# =====  
# MENSAJES DEL MAR  
# =====  
  
# Patrones de espuma observados en la orilla  
# Cada carácter representa la forma de la espuma:  
# '-' = línea recta, '/' = diagonal, 'O' = círculo  
  
patron_1 = "--//O--//O--//O"  
patron_2 = "O--O--O--O--O--"  
patron_3 = "-/O-/-/O-/-/O--"  
  
print("=== PATRONES DE ESPUMA ===")  
print(f"Patrón 1: {patron_1}")  
print(f"Patrón 2: {patron_2}")  
print(f"Patrón 3: {patron_3}")
```

Accediendo a caracteres: La posición de la ola

Cada carácter en un string tiene una posición (índice). Como cada ola en el mar tiene un orden:

```
# Accediendo a posiciones específicas  
print(f"\n--- Analizando Patrón 1 ---")
```

```
print(f"Primer carácter: '{patron_1[0]}'")
print(f"Tercer carácter: '{patron_1[2]}'")
print(f"Último carácter: '{patron_1[-1]}'")
print(f"Primeros 3: '{patron_1[0:3]}'")
print(f"Últimos 4: '{patron_1[-4:]}'")
```

Slicing: Leyendo entre olas

El **slicing** permite extraer porciones de un string. Como elegir solo ciertas olas para surfear:

```
# Slicing: [inicio:fin:paso]
print(f"\n--- Slicing de patrones ---")

# Primeros 6 caracteres
primeras_6 = patron_1[:6]
print(f"Primeras 6 posiciones: '{primeras_6}'")

# Cada 2 caracteres (saltando uno)
cada_dos = patron_1[::2]
print(f"Cada 2 posiciones: '{cada_dos}'")

# Invertir el patrón
inverso = patron_1[::-1]
print(f"Patrón invertido: '{inverso}'")
```

Métodos de string: Herramientas del surfista digital

Mateo necesitaba herramientas más potentes. Los **métodos de string** son funciones que pertenecen al string y permiten manipularlo:

```
# --- MÉTODOS DE STRING ---

mensaje_del_mar = " EL CODIGO ESTA EN LA FRECUENCIA "
```

```
# strip() - elimina espacios
limpio = mensaje_del_mar.strip()
print(f"\nOriginal: '{mensaje_del_mar}'")
print(f"Limpio: '{limpio}'")

# lower() / upper()
print(f"Minúsculas: '{limpio.lower()}'")
print(f"Mayúsculas: '{limpio.upper()}'")

# replace() - reemplazar texto
reemplazado = limpio.replace("FRECUENCIA", "ALTURA")
print(f"Reemplazado: '{reemplazado}'")

# find() - encontrar posición
pos = limpio.find("FRECUENCIA")
print(f"'FRECUENCIA' está en posición: {pos}")

# count() - contar ocurrencias
conteo_e = limpio.count("E")
```

```
print(f"Veces que aparece 'E': {conteo_e}")

# len() - longitud
print(f"Longitud del mensaje: {len(limpio)}")
```

El mensaje cifrado en las olas

Mateo transcribió los patrones de 5 olas consecutivas y notó que, al unirlos, formaban algo:

```
# --- DESCIFRANDO EL MENSAJE ---

ola_1 = "M-A-R"
ola_2 = "C-O-N"
ola_3 = "T-A-M"
ola_4 = "I-N-A"
ola_5 = "D-O--"

# Unir las olas para formar el mensaje
mensaje_completo = ola_1 + ola_2 + ola_3 + ola_4 + ola_5
print(f"\nMensaje completo: {mensaje_completo}")

# Limpiar el mensaje (quitar guiones)
mensaje_limpio = mensaje_completo.replace("-", "")
print(f"Mensaje limpio: {mensaje_limpio}")

# Convertir a minúsculas
print(f"En minúsculas: {mensaje_limpio.lower()}")

# Verificar si contiene palabras clave
if "CONTAMINACION" in mensaje_limpio.upper():
    print("¡El mensaje habla de contaminación!")
if "MAR" in mensaje_limpio.upper():
    print("¡El mensaje menciona el mar!")
```

—"MARCONTAMINADO"—leyó Mateo—. El mensaje del mar dice "mar contaminado". Alguien está enviando una advertencia.

El nombre del generador

En el parte de olas, Mateo encontró una coordenada que se repetía:

```
# --- COORDENADA DEL GENERADOR ---

coordenada = "11°46'30.7\"S 77°11'22.3\"W"
print(f"\nCoordenada del generador: {coordenada}")

# Extraer latitud y longitud
latitud = coordenada[:15]
longitud = coordenada[16:]
print(f"Latitud: {latitud}")
print(f"Longitud: {longitud}")

# El nombre del proyecto estaba codificado en la frecuencia
```

```
frecuencias = "12-15-09-21-05-14-20-15"
letras = frecuencias.split("-")
nombre_proyecto = ""
for num in letras:
    letra = chr(int(num) + 64) # A=1, B=2, etc.
    nombre_proyecto += letra
    print(f"{num} → '{letra}'")

print(f"\nNombre del proyecto: {nombre_proyecto}")
```

—"L O C U E N T O"—dijo Mateo—. "LOCUENTO". ¿Qué significa?

Buscó en Google. "Locuento" no existía. Pero "LOCUENTO" sonaba a "LOC-uento". O "LOC" como "location". O tal vez era un acrónimo.

Lo anotó y siguió investigando.

Enigmas

Enigma 2.1: Descifra la frecuencia

Las frecuencias de las olas anómalas son: "03-15-12-01-19". Convierte cada número a letra (A=1, B=2, ...) usando `chr(num + 64)` y descubre la palabra.

Enigma 2.2: Extrae el nombre del barco

Del string `"B/T-MARES-DE-ANCÓN-2026"`, extrae solo "ANCÓN" usando slicing y `find()`.

Enigma 2.3: Invertir el mensaje

El mar dejó este mensaje en la arena: `"ODATNEMIRP SE ODAGUA"`. Inviértelo con `[::-1]`. ¿Qué dice?

Enigma 2.4: Limpieza de datos

El sensor de olas registró: `"OLA: 2.4m - FREC: 12s "`. Usa `strip()`, `replace()`, y `split()` para obtener solo los valores numéricos.

Lo que aprendiste

- Los **strings** son secuencias de caracteres con posiciones (índices)
- El **slicing** `[inicio:fin:paso]` extrae partes de un string
- Los **índices negativos** cuentan desde el final
- **Métodos:** `.strip()`, `.lower()`, `.upper()`, `.replace()`, `.find()`, `.count()`
- `len()` da la longitud
- `+` concatena strings
- `[::-1]` invierte un string

Mateo tenía el mensaje: "MAR CONTAMINADO" y un nombre: "LOCUENTO". Pero necesitaba más. Necesitaba saber quién más estaba involucrado.

—Si esto es una conspiración —pensó—, debe haber más personas. Una red. Una lista.

Y para eso, necesitaba **listas** de Python.

Capítulo 3: La Lista de Implicados

Conceptos: Listas, tuplas, indexing, métodos de listas

Mateo necesitaba organizar la información. Tenía datos sueltos: nombres, lugares, frecuencias, coordenadas. Pero todo estaba desordenado.

—El mar es una lista de olas —dijo Rafa, que había llegado a ayudarlo—. Cada ola es un elemento. Tiene un orden, una posición.

—Exacto —respondió Mateo—. Y en Python, eso se llama una **lista**.

Listas: Las olas del océano de datos

Una **lista** es una colección ordenada de elementos. Como las olas en el mar: cada una tiene su lugar, su momento, su dato.

```
# =====  
# LISTA DE IMPLICADOS EN EL CASO  
# =====  
  
# Personas relacionadas al proyecto LOCUMENTO  
implicados = [  
    "Carlos Parra (Ingeniero Naval)",  
    "Dra. Luisa Rivas (Bióloga Marina)",  
    "Miguel Ángel Soto (Empresario)",  
    "Capitán Paredes (Capitanía del Puerto)",  
    "Rafa (testigo)"  
]  
  
print("=== IMPLICADOS EN EL CASO ===")  
print(implicados)  
print(f"Total de implicados: {len(implicados)}")
```

Accediendo a elementos

Cada elemento tiene una posición (índice), igual que en los strings:

```
print(f"\nPrimer implicado: {implicados[0]}")  
print(f"Último implicado: {implicados[-1]}")  
print(f"Los 3 primeros: {implicados[:3]}")  
print(f"Los 2 últimos: {implicados[-2:]}")
```

Listas de listas: Datos anidados

Cada persona tiene múltiples datos. Mateo creó una lista de listas:

```
# [nombre, edad, rol, sabe_del_generador, nivel_confianza]
fichas_implicados = [
    ["Carlos Parra", 45, "Ingeniero Naval", True, 8],
    ["Luisa Rivas", 38, "Bióloga Marina", False, 6],
    ["Miguel Ángel Soto", 62, "Empresario", True, 9],
    ["Capitán Paredes", 55, "Capitanía del Puerto", False, 4],
    ["Rafa", 29, "Amigo / Testigo", True, 10],
]

print("\n=== FICHAS COMPLETAS ===")
for ficha in fichas_implicados:
    print(f"Nombre: {ficha[0]:30} | Edad: {ficha[1]} | Confianza: {ficha[4]}/10")
```

Tuplas: Datos fijos del mar

Hay datos que no cambian. Las coordenadas geográficas, por ejemplo. Para eso están las **tuplas**:

```
# --- TUPLAS: DATOS INMUTABLES ---

# Coordenadas de Ancón (no cambian)
coordenadas_ancón = (-11.7758, -77.1897)
print(f"\nCoordenadas de Ancón: {coordenadas_ancón}")
print(f"Latitud: {coordenadas_ancón[0]}")
print(f"Longitud: {coordenadas_ancón[1]}")

# Puntos de monitoreo (tuplas fijas)
puntos_monitoreo = [
    (-11.7730, -77.1880),
    (-11.7760, -77.1900),
    (-11.7780, -77.1870),
    (-11.7740, -77.1850)
]

print(f"\nPuntos de monitoreo: {len(puntos_monitoreo)}")
for i, punto in enumerate(puntos_monitoreo, 1):
    print(f" Punto {i}: ({punto[0]}, {punto[1]})")
```

Métodos de listas: Manipulando la red

Mateo necesitaba herramientas para gestionar su lista de implicados:

```
# --- MÉTODOS DE LISTAS ---
print("\n=== MANIPULANDO LA LISTA ===")

# .append() - Agregar
implicados.append("Desconocido (operador del generador)")
print(f"Agregado: {implicados}")

# .remove() - Eliminar
```

```

implicados.remove("Rafa (testigo)")
print(f"Sin Rafa (es amigo, no implicado): {implicados}")

# .sort() - Ordenar
implicados.sort()
print(f"Ordenados alfabéticamente: {implicados}")

# .reverse() - Invertir
implicados.reverse()
print(f"Orden inverso: {implicados}")

# .pop() - Extraer el último
ultimo = implicados.pop()
print(f"Extraído: {ultimo}")

# .index() - Buscar posición
if "Carlos Parra (Ingeniero Naval)" in implicados:
    pos = implicados.index("Carlos Parra (Ingeniero Naval)")
    print(f"Carlos Parra está en posición: {pos}")

# .count() - Contar
print(f"Total actual: {len(implicados)}")

```

Evidencias del caso

Mateo organizó toda la evidencia recolectada:

```

# --- EVIDENCIAS ---

evidencias_tecnicas = [
    "Patrón de olas anómalas cada 3 horas",
    "Coordenadas del generador submarino",
    "Mensaje cifrado: 'MAR CONTAMINADO'",
    "Nombre del proyecto: LOCUENTO",
    "Frecuencias convertidas a letras"
]

testigos = [
    "Rafa (vio el generador desde su bote)",
    "Pescadores locales (oyeron ruido submarino)",
    "Capitán Paredes (recibió quejas)"
]

sospechosos_directos = [
    "Miguel Ángel Soto (financia el proyecto)",
    "Carlos Parra (diseñó el generador)",
]

print("\n=== EVIDENCIA DEL CASO ===")
print(f"\nEvidencias técnicas ({len(evidencias_tecnicas)}):")
for e in evidencias_tecnicas:
    print(f"    • {e}")

```

```

print(f"\nTestigos ({len(testigos)}):")
for t in testigos:
    print(f"    • {t}")

print(f"\nSospechosos directos ({len(sospechosos_directos)}):")
for s in sospechosos_directos:
    print(f"    • {s}")

```

El dato que cambió todo

Al revisar los datos del parte de olas, Mateo notó una lista de frecuencias que no había procesado:

```

# --- FRECUENCIAS DE OLAS ANÓMALAS ---

frecuencias_anomalas = [12, 15, 9, 21, 5, 14, 20, 15]

print(f"\nFrecuencias anómalas: {frecuencias_anomalas}")
print(f"Promedio: {sum(frecuencias_anomalas) / len(frecuencias_anomalas):.1f}")
print(f"Máxima: {max(frecuencias_anomalas)}")
print(f"Mínima: {min(frecuencias_anomalas)}")

# ¿Patrón en las frecuencias?
for i in range(len(frecuencias_anomalas) - 1):
    diferencia = frecuencias_anomalas[i+1] - frecuencias_anomalas[i]
    print(f"Diferencia entre {i+1} y {i+2}: {diferencia}")

```

El patrón era claro: las diferencias no eran constantes. No era una progresión aritmética. Era un mensaje codificado.

Enigmas

Enigma 3.1: Tu lista de playas

Crea una lista `playas\_de\_lima` con: "Ancón", "Miraflores", "Barranco", "La Punta", "Costa Verde". Luego:

- Agrega "Santa María"
- Ordena la lista
- Muestra el total
- Muestra la primera y la última

Enigma 3.2: Matriz de accesos

Crea una matriz (lista de listas) con los accesos a la playa:

```

["Ancón", True, True, "Estacionamiento"]
["Miraflores", True, False, "Parque"]
["Barranco", True, True, "Escaleras"]

```

Muestra qué playas tienen acceso para discapacitados (columna 2).

Enigma 3.3: Tuplas de coordenadas de playas

Crea una lista de tuplas con las coordenadas de 3 playas de Lima (búscalas en Google Maps o inventa). Luego itera sobre ellas mostrando: "Playa en latitud X, longitud Y".

Lo que aprendiste

- Las **listas** son colecciones ordenadas y mutables de elementos
- Se accede con índices (empiezan en 0)
 - Las **tuplas** son inmutables
 - **Métodos:** `.append()`, `.remove()`, `.sort()`, `.reverse()`, `.pop()`, `.index()`
- `.sum()`, `.max()`, `.min()` funcionan con listas numéricas

Mateo cerró el archivo. Tenía una lista de implicados, evidencias organizadas, y un patrón de frecuencias que mostraba un mensaje. Pero el mensaje completo necesitaba algo más: necesitaba relacionar cada frecuencia con una persona, un lugar, un motivo.

Necesitaba un **diccionario**.

Capítulo 4: El Diccionario del Pescador

Conceptos: Diccionarios, sets, operaciones con sets

Mateo encontró a don Eulogio, un pescador de 78 años que conocía el mar de Ancón mejor que nadie. Tenía libretas llenas de anotaciones: fechas, condiciones del mar, especies que habían desaparecido.

—Don Eulogio —le dijo Mateo—. Usted ha visto todo. ¿Qué ha cambiado en el mar?

—Todo, mijito. Las olas ya no son las mismas. Los peces se fueron. Y desde hace un mes, hay un ruido. Un zumbido que viene del fondo.

Don Eulogio abrió una libreta gastada. En ella, había organizado sus observaciones como un diccionario: cada fecha tenía una entrada con múltiples datos.

—Esto es un diccionario —dijo Mateo—. Como los de Python.

Diccionarios: La libreta del pescador

Los **diccionarios** asocian claves con valores. Como las libretas de don Eulogio: cada fecha (clave) tiene una lista de observaciones (valor).

```
# =====  
# LIBRETA DIGITAL DE DON EULOGIO  
# =====  
  
# Observaciones del mar organizadas por fecha  
observaciones = {  
    "01-06-2026": {  
        "oleaje": "moderado",  
        "viento": "sur",  
        "temperatura": 18.2,  
        "anomalias": False,  
        "especies_vistas": ["anchoveta", "lisa"]  
    },  
    "15-06-2026": {  
        "oleaje": "fuerte",  
        "viento": "sur-oeste",  
        "temperatura": 17.8,  
        "anomalias": True,  
        "especies_vistas": ["lisa"]  
    },  
    "27-06-2026": {
```

```

        "oleaje": "anómalo",
        "viento": "sur",
        "temperatura": 18.5,
        "anomalías": True,
        "especies_vistas": []
    }
}

print("=== LIBRETA DE DON EULOGIO ===")
for fecha, datos in observaciones.items():
    print(f"\nFecha: {fecha}")
    print(f" Oleaje: {datos['oleaje']}")
    print(f" Temperatura: {datos['temperatura']}°C")
    print(f" Anomalías: {datos['anomalías']}")
    print(f" Especies: {datos['especies_vistas']}")

```

Diccionario de sospechosos

Mateo creó un diccionario con todos los implicados y sus datos detallados:

```

# --- DICCIONARIO DE IMPLICADOS ---
implicados = {
    "Carlos Parra": {
        "edad": 45,
        "rol": "Ingeniero Naval",
        "involucrado": True,
        "nivel_sospecha": 8,
        "empresa": "OceanTech Perú",
        "evidencias": ["Diseñó el generador", "Recibió transferencia de Soto"]
    },
    "Luisa Rivas": {
        "edad": 38,
        "rol": "Bióloga Marina",
        "involucrado": False,
        "nivel_sospecha": 3,
        "empresa": "IMARPE (Instituto del Mar)",
        "evidencias": ["Denunció ruido submarino", "Pidió investigación"]
    },
    "Miguel Ángel Soto": {
        "edad": 62,
        "rol": "Empresario",
        "involucrado": True,
        "nivel_sospecha": 9,
        "empresa": "Grupo Inmobiliario Costa Azul",
        "evidencias": ["Dueño del proyecto LOCUMENTO", "Quiere construir en Ancón"]
    },
    "Capitán Paredes": {
        "edad": 55,
        "rol": "Capitanía del Puerto",
        "involucrado": False,
        "nivel_sospecha": 5,
        "empresa": "Gobierno del Perú",
        "evidencias": ["Recibió quejas", "No investigó"]
    }
}

```

```

    }
}

print("\n=== FICHAS DE IMPLICADOS ===")
for nombre, datos in implicados.items():
    print(f"\n {nombre} ({datos['edad']})")
    print(f"    Rol: {datos['rol']}")
    print(f"    Sospecha: {datos['nivel_sospecha']/10}")
    print(f"    Evidencias: {'', '.join(datos['evidencias'])}")

```

Métodos de diccionarios

```

# --- MÉTODOS ---
print("\n=== CONSULTAS ===")

# keys()
nombres = list(implicados.keys())
print(f"Implicados: {nombres}")

# values()
print("\nRoles:")
for datos in implicados.values():
    print(f"    • {datos['rol']} - {datos['empresa']}")

# Verificar existencia
if "Carlos Parra" in implicados:
    print(f"\nCarlos Parra está fichado")

# .get() con valor por defecto
sospecha = implicados.get("Rafa", {}).get("nivel_sospecha", 0)
print(f"Nivel de sospecha de Rafa (si existiera): {sospecha}")

```

Sets: Especies únicas del mar

Para analizar qué especies habían desaparecido, Mateo usó **sets** —colecciones de elementos únicos:

```

# --- SETS: ESPECIES ÚNICAS ---

especies_por_fecha = {
    "01-06": {"anchoveta", "lisa", "caballa"},
    "15-06": {"lisa", "caballa"},
    "20-06": {"lisa"},
    "27-06": set()
}

print("\n=== ANÁLISIS DE ESPECIES ===")

# Especies que había al inicio
inicio = especies_por_fecha["01-06"]
print(f"Especies al inicio: {inicio}")

```

```

# Especies que había al final
final = especies_por_fecha["27-06"]
print(f"Especies al final: {final}")

# Especies desaparecidas
desaparecidas = inicio - final
print(f"Especies desaparecidas: {desaparecidas}")

# Especies que se mantuvieron (intersección de todas)
comunes = especies_por_fecha["01-06"]
for fecha in ["15-06", "20-06", "27-06"]:
    comunes = comunes & especies_por_fecha[fecha]
print(f"Especies presentes siempre: {comunes}")

# Todas las especies vistas alguna vez
todas = set()
for especies in especies_por_fecha.values():
    todas = todas | especies
print(f"Todas las especies vistas: {todas}")

```

El proyecto LOCUMENTO

Mateo encontró los papeles del proyecto LOCUMENTO en la oficina del puerto. Los datos estaban en un diccionario:

```

# --- PROYECTO LOCUMENTO ---

proyecto_locumento = {
    "nombre_completo": "LOCalizador de Corrientes y Oleaje para Urbanización Nortina",
    "empresa": "Grupo Inmobiliario Costa Azul",
    "presupuesto": 2500000,
    "etapas": ["Estudio", "Instalación", "Pruebas", "Operación"],
    "ubicacion": "Ancón, 500m mar adentro",
    "equipo": {
        "ingeniero_jefe": "Carlos Parra",
        "financiista": "Miguel Ángel Soto",
        "operador": "Desconocido"
    },
    "estado": "Pruebas"
}

print("\n=== PROYECTO LOCUMENTO ===")
for clave, valor in proyecto_locumento.items():
    if isinstance(valor, dict):
        print(f"\n{clave}:")
        for k, v in valor.items():
            print(f"  {k}: {v}")
    else:
        print(f"{clave}: {valor}")

```

Enigmas

Enigma 4.1: Tu diccionario de playa

Crea un diccionario ``mi_playa`` con: `nombre`, `ubicación`, `temperatura_agua`, `tiene_estacionamiento`, y `ola_favorita`. Luego muestra cada dato.

Enigma 4.2: Consulta de implicados

Del diccionario ``implicados``, muestra:

- Los nombres de todos
- Quién tiene `nivel_sospecha > 5`
- Las evidencias de Carlos Parra

Enigma 4.3: Sets de deportes acuáticos

Dados estos sets:

```
surfistas = {"Mateo", "Rafa", "Lucía"}
buceadores = {"Rafa", "Pedro", "Lucía"}
kayakistas = {"Mateo", "Pedro", "Sofía"}
```

Encuentra:

- Quiénes hacen surf y buceo (intersección)
 - Quiénes hacen solo surf (diferencia)
 - Todos los deportistas (unión)
-

Lo que aprendiste

- Los **diccionarios** asocian claves con valores ``{clave: valor}``
- Se accede con ``diccionario[clave]``
 - **Métodos:** ``.keys()``, ``.values()``, ``.items()``, ``.get()``
- Los **sets** son colecciones de elementos únicos
- Los diccionarios pueden anidarse

Mateo tenía el proyecto LOCUMENTO mapeado. Pero había un problema: el "operador" del generador aparecía como "Desconocido". Alguien más estaba involucrado. Alguien que no aparecía en los papeles.

—Necesito saber quién opera el generador —dijo—. Y para eso, necesito tomar decisiones. Evaluar a cada implicado. Si hizo X, entonces Y.

Necesitaba **condicionales**.

Capítulo 5: La Decisión de la Ola

Conceptos: Condicionales `if/elif/else`, operadores lógicos

Mateo estaba en la orilla, mirando el mar. El generador submarino seguía activo. Las olas seguían siendo anómalas. Pero había decidido algo: esa noche iba a bucear hasta el generador para desactivarlo.

—Si vas solo, estás loco —dijo Rafa.

—Si no voy, el proyecto LOCUMENTO sigue adelante y privatizan Ancón.

—¿Y si te atrapan?

—Entonces necesito un plan B.

Mateo abrió su laptop. Necesitaba evaluar escenarios. Y en Python, los escenarios se evalúan con **condicionales**.

If/Elif/Else: La decisión del surfista

Los condicionales permiten que el código tome decisiones. Como un surfista decidiendo si tomar una ola o esperar la siguiente:

```
# =====
# EVALUANDO ESCENARIOS
# =====

altura_ola = 2.4
experiencia_surfista = 7 # años

print("=== DECISIÓN: ¿TOMAMOS LA OLA? ===\n")

if altura_ola < 1.0:
    print("Ola muy pequeña. No vale la pena.")
elif altura_ola <= 2.0:
    print("Ola moderada. Buena para practicar.")
elif altura_ola <= 3.5:
    if experiencia_surfista >= 5:
        print("Ola perfecta para surfista experimentado. ¡A remar!")
    else:
        print("Ola grande para principiante. Mejor esperar.")
else:
    print("Ola peligrosa. No entrar al mar.")
```

Operadores de comparación

Operador	Significado
`==`	Igual a
`!=`	Diferente de
`<`	Menor que
`>`	Mayor que
`<=`	Menor o igual
`>=`	Mayor o igual

```
# --- EVALUANDO IMPLICADOS ---

# Diccionario de implicados (del capítulo anterior)
implicados = {
    "Carlos Parra": {"edad": 45, "rol": "Ingeniero Naval", "involucrado": True, "nivel_sospecha": 8, "empresa": "Petrobras"},
    "Luisa Rivas": {"edad": 38, "rol": "Bióloga Marina", "involucrado": False, "nivel_sospecha": 3, "empresa": "Petrobras"},
    "Miguel Ángel Soto": {"edad": 62, "rol": "Empresario", "involucrado": True, "nivel_sospecha": 9, "empresa": "Petrobras"},
    "Capitán Paredes": {"edad": 55, "rol": "Capitanía del Puerto", "involucrado": False, "nivel_sospecha": 5, "empresa": "Petrobras"}
}

print("\n=== EVALUACIÓN DE IMPLICADOS ===\n")

for nombre, datos in implicados.items():
    print(f" {nombre}")

    if datos["nivel_sospecha"] >= 8:
        print(f"   ALTO: Sospechoso principal")
    elif datos["nivel_sospecha"] >= 5:
        print(f"   MEDIO: Requiere vigilancia")
    else:
        print(f"   BAJO: Probablemente inocente")
```

Operadores lógicos: and, or, not

Las decisiones reales usan múltiples condiciones:

```
# --- ANÁLISIS MULTIFACTOR ---

for nombre, datos in implicados.items():
    print(f"\n {nombre}")

    # and: ambas condiciones
    if datos["involucrado"] and datos["nivel_sospecha"] >= 7:
        print(f"   IMPLICADO DIRECTO: Involucrado y muy sospechoso")

    # or: al menos una
    if datos["nivel_sospecha"] >= 8 or "Soto" in datos.get("empresa", ""):
        print(f"   POSIBLE CABECILLA: Sospecha máxima o conexión con Soto")

    # not: negación
    if not datos["involucrado"]:
        print(f"   Aparentemente limpio")
```


El plan de acción

Mateo escribió un sistema para decidir su plan según las condiciones del mar:

```
# --- SISTEMA DE DECISIÓN PARA EL BUCEO ---

def evaluar_buceo(velocidad_viento, altura_ola, visibilidad, hay_luna):
    """Evalúa si es seguro bucear hasta el generador."""

    print("\n=== EVALUACIÓN DE BUCEO NOCTURNO ===")
    print(f"Viento: {velocidad_viento} nudos")
    print(f'Olas: {altura_ola}m")
    print(f'Visibilidad: {visibilidad}m")
    print(f'Luna: {'Sí' if hay_luna else 'No'}")

    riesgos = 0

    if velocidad_viento > 15:
        print("    Viento fuerte")
        riesgos += 2
    if altura_ola > 1.5:
        print("    Olas grandes")
        riesgos += 2
    if visibilidad < 3:
        print("    Mala visibilidad")
        riesgos += 3
    if not hay_luna:
        print("    Sin luna (muy oscuro)")
        riesgos += 1

    print(f"\n  Total de riesgos: {riesgos}/10")

    if riesgos >= 7:
        return "NO RECOMENDADO: Condiciones peligrosas"
    elif riesgos >= 4:
        return "PRECAUCIÓN: Posible pero con equipo adecuado"
    else:
        return "FAVORABLE: Condiciones seguras"

# Evaluar con condiciones actuales
resultado = evaluar_buceo(12, 1.2, 4, True)
print(f"\n  → Resultado: {resultado}")
```

El interrogatorio digital

Mateo decidió que necesitaba hablar con Carlos Parra, el ingeniero naval. Pero antes quería preparar las preguntas basadas en los datos:

```
# --- PREPARANDO EL INTERROGATORIO ---

def analizar_respuesta(respuesta, evidencia_conocida):
    """Analiza si la respuesta coincide con la evidencia."""
```

```

if respuesta == evidencia_conocida:
    return "VERDAD: Coincide con la evidencia"
elif respuesta.lower() == "no sé" or respuesta.lower() == "no recuerdo":
    return "EVASIVO: Podría estar ocultando algo"
elif respuesta != evidencia_conocida:
    return f"MENTIRA: Dice '{respuesta}' pero la evidencia muestra '{evidencia_conocida}'"
else:
    return "INDETERMINADO"

# Simular respuestas de Carlos Parra
preguntas = [
    ("¿Diseñaste el generador submarino?", "Sí, pero era para investigación"),
    ("¿Sabes quién lo opera?", "No sé"),
    ("¿Recibiste dinero de Miguel Ángel Soto?", "No, fue una inversión legítima"),
]

evidencias = [
    "Sí, diseñó el generador",
    "Los registros muestran que sí sabe",
    "Transferencia bancaria comprobada"
]

print("\n=== ANÁLISIS DE RESPUESTAS ===")
for (pregunta, respuesta), evidencia in zip(preguntas, evidencias):
    print(f"\nP: {pregunta}")
    print(f"R: '{respuesta}'")
    print(f"E: {evidencia}")
    print(f"→ {analizar_respuesta(respuesta, evidencia)}")

```

Enigmas

Enigma 5.1: Evaluador de olas

Pide al usuario la altura de una ola (con `input()`). Evalúa:

- Si es < 1m: "Ola pequeña"
- Si es entre 1 y 2.5m: "Ola ideal para surf"
- Si es > 2.5m: "Ola peligrosa"

Enigma 5.2: Filtro de sospechosos

Del diccionario ``implicados``, muestra solo los que:

- Tengan `nivel_sospecha >= 7`
- Estén involucrados Y tengan evidencias
- No estén involucrados PERO tengan `nivel_sospecha > 4`

Enigma 5.3: El semáforo del mar

Escribe un programa que, dada la temperatura del agua, muestre:

- < 16°C: "Agua muy fría, usa traje de neopreno"
- 16-20°C: "Agua templada, buenas condiciones"

- `> 20°C`: "Agua cálida, ideal para todo el día"
-

Lo que aprendiste

- ``if`, `elif`, `else`` ejecutan código según condiciones
 - **Operadores de comparación:** ``==`, `!=`, `<`, `>`, `<=`, `>=``
 - **Operadores lógicos:** ``and`, `or`, `not``
- Los condicionales pueden anidarse
- Las condiciones múltiples permiten análisis complejos

Mateo tenía su plan. Las condiciones del mar eran favorables. Iría esa noche.

Pero cuando se disponía a guardar su laptop, recibió un mensaje de texto de un número desconocido:

```
"Sabemos que estás investigando.  
No vayas al generador esta noche.  
No es seguro.  
— Alguien que quiere ayudarte."
```

Mateo miró el mensaje. ¿Era una advertencia real o una trampa?

—Si voy, puedo descubrir la verdad —dijo—. Si no voy, quizás pierdo la única oportunidad.

A veces, la decisión correcta no es la más obvia. A veces, hay que recorrer un **camino**, paso a paso, como un **bucle**.

Capítulo 6: El Viaje de Mateo

Conceptos: Bucles ``for`/`while``, ``range()``, ``enumerate()``, ``break`/`continue``

Eran las 11:00 p.m. cuando Mateo llegó al muelle de Ancón. La luna estaba alta. El mar estaba tranquilo. Pero en el horizonte, unas luces parpadeaban.

—Es el generador —dijo Rafa, que lo acompañaba—. Está a 500 metros mar adentro.

—Necesito llegar hasta él. Pero no en línea recta. Hay boyas de monitoreo. Si paso cerca de ellas, activo una alarma.

—¿Cómo piensas llegar entonces?

Mateo señaló un patrón en el agua.

—Las olas van en una dirección. El generador está en otra. Pero hay un camino. Como un bucle en Python: paso a paso, evitando los obstáculos.

For: Navegando entre olas

El bucle ``for`` recorre una secuencia de elementos. Como navegar de boya en boya:

```
# =====  
# RUTA HACIA EL GENERADOR  
# =====  
  
# Puntos de referencia en el mar (boyas)  
puntos_ruta = [  
    "Muelle de Ancón",  
    "Boya Roja #1",  
    "Roca del Norte",  
    "Boya Verde #2",  
    "Banco de Arena",  
    "Generador Submarino"  
]  
  
print("=== RUTA HACIA EL GENERADOR ===\n")  
print("Recorriendo puntos de navegación:\n")  
  
for punto in puntos_ruta:  
    print(f" → Avanzando hacia: {punto}")  
    if punto == "Generador Submarino":  
        print(f" ¡Destino alcanzado!")
```

range(): Distancias numéricas

A veces necesitas contar. Como los metros que nada Mateo:

```
print("\n=== CONTANDO METROS NADADOS ===\n")

for metro in range(1, 11):
    print(f" Metro {metro}: braceo...")

print(" ¡Descanso! Llegué a los 10 metros.")
```

enumerate(): Posición y valor

`enumerate()` da el índice y el valor al mismo tiempo:

```
print("\n=== BOYAS CON SUS COORDENADAS ===\n")

boyas = [
    "Boya Roja (inicio)",
    "Boya Azul (norte)",
    "Boya Verde (este)",
    "Boya Amarilla (generador)"
]

for indice, boya in enumerate(boyas):
    print(f" Boya [{indice}]: {boya}")
```

While: Hasta encontrar el generador

El bucle `while` se ejecuta mientras una condición sea verdadera. Como Mateo nadando hasta alcanzar el generador:

```
print("\n=== NADANDO HACIA EL GENERADOR ===\n")

distancia = 500 # metros
metros_nadados = 0

while metros_nadados < distancia:
    metros_nadados += 25 # brazadas de 25 metros
    progreso = (metros_nadados / distancia) * 100
    print(f" Progreso: {metros_nadados}/{distancia}m ({progreso:.0f}%)")
    time.sleep(0.2)

print(f"\n ¡Generador alcanzado después de nadar {metros_nadados} metros!")
```

break: Evadiendo la patrulla

De repente, Mateo vio luces. Una patrulla de la Capitanía se acercaba. Necesitaba salir del bucle:

```
print("\n=== EVADIENDO A LA PATRULLA ===\n")

ruta_segura = ["bajo el muelle", "detrás de la roca", "zona de boyas", "junto al generador"]
patrulla_cerca = False
```

```

for zona in ruta_segura:
    if patrulla_cerca:
        print(f"   Patrulla detectada en {zona}. ¡BREAK! Ocultándose...")
        break

    print(f" → Avanzando por {zona}...")

    # Simular detección de patrulla en la tercera zona
    if zona == "zona de boyas":
        print(f"   ¡Luces de patrulla a lo lejos!")
        patrulla_cerca = True
        print(f" → Rompiendo ruta. Parando en {zona}.")
        break

print(f"\n → Búsqueda {'interrumpida por patrulla' if patrulla_cerca else 'completada'}.")

```

continue: Ignorando obstáculos menores

Algunos obstáculos no requieren detenerse, solo evitarlos:

```

print("\n=== SORTEANDO OBSTÁCULOS MENORES ===\n")

obstaculos = ["alga", "medusa", "roca", "red", "plástico", "generador"]

for obstaculo in obstaculos:
    if obstaculo in ["alga", "medusa", "plástico"]:
        print(f" → {obstaculo} detectado. Esquivando (continue)...")
        continue

    print(f" → {obstaculo}: Bajando a revisar...")

    if obstaculo == "generador":
        print(f"   ¡GENERADOR ENCONTRADO!")
        break

```

El registro de activaciones

Mateo llegó al generador y encontró una pequeña computadora sellada en una caja estanca. Descargó el registro de activaciones para analizarlo después:

```

# --- REGISTRO DE ACTIVACIONES DEL GENERADOR ---

print("\n=== REGISTRO DE ACTIVACIONES ===\n")

activaciones = [
    {"fecha": "01-06", "hora": "06:00", "duracion": 15},
    {"fecha": "01-06", "hora": "09:00", "duracion": 15},
    {"fecha": "01-06", "hora": "12:00", "duracion": 15},
    {"fecha": "01-06", "hora": "15:00", "duracion": 15},
    {"fecha": "01-06", "hora": "18:00", "duracion": 15},
    {"fecha": "01-06", "hora": "21:00", "duracion": 15},
    {"fecha": "01-06", "hora": "00:00", "duracion": 30},
    {"fecha": "02-06", "hora": "03:00", "duracion": 30},

```

```

]

print("Activaciones registradas:")
for act in activaciones:
    print(f"  {act['fecha']} | {act['hora']} | {act['duracion']} min")

# Análisis: buscar patrones en las activaciones
print("\n=== ANÁLISIS DE PATRONES ===\n")

for i, act in enumerate(activaciones):
    if i > 0:
        anterior = activaciones[i-1]
        if act["duracion"] != anterior["duracion"]:
            print(f"  Cambio de duración: {anterior['duracion']} → {act['duracion']} min a las {act['hora']}h")

# ¿Cuánto tiempo total de activación?
total_minutos = sum(act["duracion"] for act in activaciones)
print(f"\n  Tiempo total activado: {total_minutos} minutos")
print(f"  Equivalente a: {total_minutos // 60}h {total_minutos % 60}m")

```

El patrón era claro: activaciones cada 3 horas, con duraciones que variaban. Como un programa con diferentes configuraciones.

Enigmas

Enigma 6.1: Recorrido de playas

Dada la lista `playas = ["Ancón", "Miraflores", "Barranco", "Costa Verde"]`, usa `for` para mostrar cada playa con su número (usando `enumerate`).

Enigma 6.2: Búsqueda del generador

Usando `while`, simula una búsqueda del generador. Parte de 0 metros, avanza de 10 en 10, y cuando llegues a 100 metros, muestra "GENERADOR ENCONTRADO".

Enigma 6.3: Filtro de datos erróneos

Dada la lista de temperaturas del mar: `[18.2, 19.0, -5, 18.5, 500, 19.1]`, usa `continue` para saltar los valores imposibles (negativos o > 50). Muestra solo los válidos.

Enigma 6.4: Pirámide de olas

Usando `range()` y bucles anidados, crea:

```

*
**
***
****

```

Lo que aprendiste

- `for elemento in coleccion:` recorre una secuencia
- `range(n)` genera números de 0 a n-1

- `enumerate()` da índice y valor
- `while condicion:` itera mientras se cumpla la condición
- `break` sale del bucle
- `continue` salta a la siguiente iteración

Mateo tenía el registro de activaciones. Ahora sabía que el generador no solo creaba olas anómalas: también transmitía datos. Cada activación era una función del programa principal.

—El generador no es solo una máquina —dijo—. Es un programa ejecutándose en el mar. Y cada función del programa hace algo diferente. Necesito entender esas **funciones**.

Capítulo 7: Las Funciones del Mar

Conceptos: Funciones, parámetros, `return`, alcance de variables

Mateo había logrado descargar el firmware del generador. Era un archivo Python. Alguien había programado todo el sistema en Python.

—Es hermoso —dijo, mientras abría el archivo—. El generador tiene funciones. Cada función controla un aspecto de las olas.

—¿Como qué? —preguntó Rafa.

—Como `generar\_ola(altura, frecuencia)` o `enviar\_mensaje(texto)`. El código está organizado en bloques reutilizables. Como las olas: cada una es única, pero todas siguen el mismo patrón.

Funciones: El ritual del surfista

Una **función** es un bloque de código que realiza una tarea específica y puede reutilizarse. Como el ritual de un surfista antes de remar: siempre los mismos pasos, adaptados a cada ola.

```
# =====
# FUNCIONES DEL GENERADOR
# =====

# Función básica sin parámetros
def mostrar_bienvenida():
    print("=" * 40)
    print("GENERADOR DE OLAS - SISTEMA LOCUENTO")
    print("=" * 40)

mostrar_bienvenida()
```

Parámetros: Las variables de la ola

Los **parámetros** son los datos que recibe la función. Como la altura y frecuencia de una ola:

```
def generar_ola(altura, frecuencia):
    """Genera una ola con los parámetros especificados."""
    print(f"\n    OLA GENERADA:")
    print(f"    Altura: {altura} metros")
    print(f"    Frecuencia: {frecuencia} segundos")
    print(f"    Tipo: {'Anómala' if altura > 2.0 else 'Normal'}")
```

```

generar_ola(1.8, 12)
generar_ola(2.4, 8)
generar_ola(0.5, 20)

```

Parámetros por defecto

Algunos parámetros tienen valores que se usan la mayoría del tiempo:

```

def crear_parte_olas(playa="Ancón", altura_promedio=1.5, temperatura=18.0):
    """Crea un parte de olas con valores por defecto."""
    print(f"\n=== PARTE DE OLAS: {playa.upper()} ===")
    print(f"Altura promedio: {altura_promedio}m")
    print(f"Temperatura: {temperatura}°C")
    print(f"Recomendación: {'Ideal para surf' if altura_promedio >= 1.0 and altura_promedio <= 2.5 else 'P")

crear_parte_olas()
crear_parte_olas("Miraflores", 2.0, 19.5)
crear_parte_olas("Punta Hermosa", altura_promedio=2.8)

```

Return: El resultado del mar

Las funciones no solo hacen cosas: también **devuelven** resultados:

```

def analizar_ola(altura, frecuencia, direccion):
    """Analiza una ola y devuelve su clasificación."""

    if altura > 2.0 and frecuencia < 10:
        return "OLA ANÓMALA - Generada artificialmente"
    elif altura > 1.5:
        return "OLA GRANDE - Condición natural"
    elif altura > 0.8:
        return "OLA MODERADA - Buena para surf"
    else:
        return "OLA PEQUEÑA - No recomendada"

# Usar la función
resultado_1 = analizar_ola(2.4, 8, "SO")
resultado_2 = analizar_ola(1.2, 15, "SO")

print(f"\nOla 1: {resultado_1}")
print(f"Ola 2: {resultado_2}")

```

Múltiples valores de retorno

Una función puede devolver varios valores como tupla:

```

def estadisticas_olas(alturas):
    """Calcula estadísticas de una lista de alturas de olas."""
    total = len(alturas)
    promedio = sum(alturas) / total if total > 0 else 0
    maxima = max(alturas) if alturas else 0
    minima = min(alturas) if alturas else 0

```

```

        return total, promedio, maxima, minima

# Desempaquetar el resultado
alturas_semana = [1.2, 1.8, 2.4, 1.5, 0.9, 2.1, 1.6]
total, prom, maxi, mini = estadisticas_olas(alturas_semana)

print(f"\n=== ESTADÍSTICAS SEMANALES ===")
print(f"Total de olas: {total}")
print(f"Altura promedio: {prom:.1f}m")
print(f'Ola más alta: {maxi}m')
print(f'Ola más baja: {mini}m')

```

Las funciones del generador LOCUENTO

Mateo encontró las funciones reales del generador en el firmware:

```

# --- FIRMWARE DEL GENERADOR LOCUENTO ---

def generar_ola_anomala(altura, frecuencia, codigo):
    """Genera una ola anómala que codifica un mensaje."""
    ola = {
        "altura": altura,
        "frecuencia": frecuencia,
        "codigo": codigo,
        "tipo": "anómala"
    }
    print(f"    Transmitiendo código '{codigo}' en ola de {altura}m")
    return ola

def codificar_mensaje(texto):
    """Convierte un texto en frecuencias de olas."""
    frecuencias = []
    for letra in texto.upper():
        if letra.isalpha():
            codigo = ord(letra) - 64 # A=1, B=2
            frecuencias.append(codigo)
    return frecuencias

def transmitir_mensaje(mensaje, altura_base=1.8):
    """Transmite un mensaje completo usando olas anómalas."""
    print(f"\n=== TRANSMITIENDO: '{mensaje}' ===\n")
    frecuencias = codificar_mensaje(mensaje)

    olas_transmitidas = []
    for i, codigo in enumerate(frecuencias):
        altura = altura_base + (i * 0.1)
        ola = generar_ola_anomala(altura, codigo, codigo)
        olas_transmitidas.append(ola)

    return olas_transmitidas

# Probar la transmisión

```

```
mensaje_secreto = "LOCUENTO"
olas = transmitir_mensaje(mensaje_secreto)
print(f"\nTotal de olas transmitidas: {len(olas)}")
```

Alcance de variables: El mundo de cada función

Las variables dentro de una función no existen fuera de ella:

```
# Variable global
playa_principal = "Ancón"

def cambiar_playa():
    # Variable local
    playa_temporal = "Miraflores"
    print(f"Dentro de la función: playa temporal = {playa_temporal}")

    # Para modificar la global, necesitamos 'global'
    global playa_principal
    playa_principal = "Punta Hermosa"
    print(f"Dentro de la función: global cambiada a {playa_principal}")

cambiar_playa()
print(f"Fuera de la función: playa principal = {playa_principal}")

# print(playa_temporal) # ¡Error! No existe fuera
```

Enigmas

Enigma 7.1: Tu primera función

Escribe `recomendar_traje(temp)` que reciba la temperatura del agua y devuelva:

- Si `temp < 16`: "Traje de neopreno 5/4mm"
- Si `16 <= temp <= 20`: "Traje corto"
- Si `temp > 20`: "Solo shorts"

Enigma 7.2: Calculadora de distancia

Crea `distancia_buceo(minutos, ritmo=10)` donde `ritmo` es metros por minuto. Devuelve la distancia recorrida.

Enigma 7.3: Valores por defecto

Crea una función `alerta_ola(altura=1.5, peligro=False)` que muestre un mensaje diferente si `peligro` es True.

Enigma 7.4: Múltiples retornos

Crea una función `info_playa(nombre, tiene_muelle, tiene_estacionamiento)` que devuelva una tupla con la puntuación (suma de facilities) y la recomendación.

Lo que aprendiste

- ``def nombre_funcion():`` define una función
- Los **parámetros** son los datos de entrada
- **Valores por defecto** hacen parámetros opcionales
- ``return`` devuelve uno o más valores
- **Ámbito (scope)**: variables locales vs. globales
- ``global`` modifica variables globales dentro de funciones

Mateo había descubierto cómo funcionaba el generador. Cada ola anómala era una función ejecutándose. Cada mensaje era un conjunto de datos transmitidos.

—Pero esto no es todo —dijo—. El generador guarda registros. Bitácoras. Archivos con datos históricos. Si puedo acceder a esos **archivos**, puedo encontrar quién programó todo esto.

Capítulo 8: Los Archivos del Puerto

Conceptos: Manejo de archivos, `open()`, `with`, `read`/`write`

Mateo tenía el firmware del generador. Pero necesitaba más: necesitaba los registros históricos del proyecto LOCUMENTO. Esos registros estaban en la computadora de la Capitanía del Puerto.

—Rafa —dijo—. ¿Puedes conseguirme los archivos?

—Estás loco. Esa información es clasificada.

—El mar no tiene clasificaciones, Rafa. El mar es de todos.

Rafa suspiró y asintió. Esa noche, le envió un archivo cifrado.

Leyendo archivos: Abriendo el mar

En Python, `open()` abre un archivo. El bloque `with` asegura que se cierre correctamente:

```
# =====
# LECTURA DE ARCHIVOS DEL PUERTO
# =====

# Crear un archivo de ejemplo con datos del puerto
datos_puerto = """BITÁCORA DEL PUERTO DE ANCÓN
FECHA: 27-06-2026

EMBARCACIONES REGISTRADAS:
- B/T "Mares del Sur" - Pesquero - 15:30
- Yate "Costa Azul" - Privado - 16:45
- Bote "Don Eulogio" - Pesquero artesanal - 05:00

ACTIVIDADES SOSPECHOSAS:
- 22:00 - Ruido submarino en sector norte
- 23:30 - Luces intermitentes en zona de boyas
- 01:00 - Embarcación no identificada

NOTA: Se recomienda no investigar.
"""

with open("bitacora_puerto.txt", "w", encoding="utf-8") as f:
    f.write(datos_puerto)

# Leer el archivo completo
with open("bitacora_puerto.txt", "r", encoding="utf-8") as f:
```

```

    contenido = f.read()

print("=== BITÁCORA DEL PUERTO ===")
print(contenido)

```

Leyendo línea por línea

```

print("\n=== LEYENDO LÍNEA POR LÍNEA ===")
with open("bitacora_puerto.txt", "r", encoding="utf-8") as f:
    for num, linea in enumerate(f, 1):
        print(f"{num:3}: {linea.strip()}")

```

Diferentes modos de apertura

Modo	Significado
"r"	Lectura
"w"	Escritura (sobrescribe)
"a"	Añadir (append)
"r+"	Lectura y escritura

Procesando el archivo de embarcaciones

Mateo encontró un archivo CSV con las embarcaciones que habían estado cerca del generador:

```

# --- PROCESANDO CSV DE EMBARCACIONES ---

csv_embarcaciones = """nombre,tipo,horario,dias_activo
Mares del Sur,Pesquero,15:30-23:00,30
Costa Azul,Yate privado,16:00-02:00,45
Don Eulogio,Pesquero artesanal,05:00-12:00,60
Lancha Rápida,No identificada,23:00-03:00,15
"""

with open("embarcaciones.csv", "w", encoding="utf-8") as f:
    f.write(csv_embarcaciones)

print("\n=== EMBARCACIONES SOSPECHOSAS ===\n")

with open("embarcaciones.csv", "r", encoding="utf-8") as f:
    encabezados = f.readline().strip().split(",")
    print(f"Columnas: {encabezados}\n")

    for linea in f:
        datos = linea.strip().split(",")
        nombre, tipo, horario, dias = datos
        print(f"    • {nombre:25} | {tipo:25} | {horario:15} | {dias} días activo")

    # Detectar embarcaciones sospechosas
    if int(dias) < 20 and "no identificada" in tipo.lower():
        print(f"        ¡SOSPECHOSA! Poco tiempo activo y no identificada")

```

```
# Lectura con list comprehension
print("\n=== EMBARCACIONES NO IDENTIFICADAS ===")
with open("embarcaciones.csv", "r", encoding="utf-8") as f:
    next(f) # Saltar encabezados
    no_id = [linea.strip().split(",") for linea in f if "no identificada" in linea.lower()]

for emb in no_id:
    print(f" → {emb[0]} - Horario: {emb[2]}")
```

Escribiendo evidencia

Mateo documentó todo lo que había descubierto:

```
# --- DIARIO DE INVESTIGACIÓN ---

print("\n=== DOCUMENTANDO EL CASO ===\n")

with open("diario_mateo.txt", "w", encoding="utf-8") as d:
    d.write("=== DIARIO DE INVESTIGACIÓN ==\n")
    d.write("Caso: Proyecto LOCUMENTO - Contaminación de olas en Ancón\n")
    d.write("Investigador: Mateo Sánchez\n\n")
    d.write("--- DÍA 1 ---\n")
    d.write("Encontré patrones anómalos en las olas de Ancón.\n")
    d.write("Descubrí el generador submarino.\n")
    d.write("El firmware del generador tiene funciones en Python.\n")

# Verificar
with open("diario_mateo.txt", "r", encoding="utf-8") as d:
    print(d.read())

# Agregar más datos
print("=== AGREGANDO EVIDENCIA ===\n")
with open("diario_mateo.txt", "a", encoding="utf-8") as d:
    d.write("\n--- DÍA 2 ---\n")
    d.write("Identifiqué a los implicados.\n")
    d.write("El proyecto LOCUMENTO es una fachada para privatizar Ancón.\n")
    d.write("El operador del generador sigue siendo desconocido.\n")

with open("diario_mateo.txt", "r", encoding="utf-8") as d:
    print(d.read())
```

El archivo del operador

Entre los archivos, Mateo encontró uno con un nombre extraño:

```
# --- ARCHIVO DEL OPERADOR ---

with open("operador.log", "w", encoding="utf-8") as f:
    f.write("""OPERADOR: DESCONOCIDO
ÚLTIMO ACCESO: 27-06-2026 23:45
FIRMA DIGITAL: 4a8f2c1e9d3b7a0c5f6e
```



```

COMANDOS EJECUTADOS:
- generar_ola(2.4, 8, "LOCUENTO")
- transmitir_mensaje("MAR CONTAMINADO")
- generar_ola(1.8, 12, "SOS")
- transmitir_mensaje("AYUDA")
""")

print("=== ARCHIVO DEL OPERADOR ===\n")

with open("operador.log", "r", encoding="utf-8") as f:
    lineas = f.readlines()

for linea in lineas:
    linea = linea.strip()
    print(f" {linea}")

# Buscar comandos sospechosos
print("\n=== COMANDOS SOSPECHOSOS ===")
with open("operador.log", "r", encoding="utf-8") as f:
    for linea in f:
        if "transmitir_mensaje" in linea:
            print(f" {linea.strip()}")

```

Enigmas

Enigma 8.1: Crea tu bitácora

Usando `open()` y `with`, crea un archivo `mi_diario_mar.txt` con la fecha de hoy y un párrafo sobre lo que aprendiste. Luego léelo.

Enigma 8.2: Procesa el archivo de temperaturas

Dado un archivo con temperaturas del mar por día (una por línea), lee el archivo y calcula el promedio.

Enigma 8.3: Agregando datos

Crea un programa que:

1. Pregunte "¿Qué viste en el mar hoy?" (input)
2. Agregue esa observación al final de `observaciones_mar.txt` (modo append)
3. Muestre todo el contenido

Enigma 8.4: Buscador en archivos

Escribe un programa que lea `bitacora_puerto.txt` y cuente cuántas veces aparece la palabra "sospechosa".

Lo que aprendiste

- `open(archivo, modo)` abre un archivo
- El bloque `with` cierra automáticamente
 - **Modos:** `"r"` (lectura), `"w"` (escritura), `"a"` (append)

- `.read()` lee todo, se puede iterar línea por línea
- Archivos CSV se procesan con `.split(",")`

Mateo encontró algo inquietante en el archivo del operador: el último comando era `transmitir_mensaje("AYUDA")`. Alguien estaba pidiendo ayuda. No era el generador transmitiendo un mensaje de prueba: era el operador.

—El operador no es parte de la conspiración —dijo Mateo—. Es un rehén. O está siendo forzado a operar el generador.

—¿O alguien que quiere ser descubierto? —sugirió Rafa.

—Hay una forma de saberlo. Forzar un error en el sistema. Porque cuando un programa falla, revela información que no debería.

Capítulo 9: El Error en la Red

Conceptos: Manejo de errores, `try`/`except`/`finally`, excepciones personalizadas

Mateo había descargado el software del generador. Pero cuando intentó ejecutar una simulación, el programa falló. Y en el fallo, encontró una pista.

```
Traceback (most recent call last):
  File "generador.py", line 42, in transmitir_mensaje
    ola = generar_ola(altura, frecuencia, codigo_morse)
NameError: name 'codigo_morse' is not defined
```

—El error revela que hay una variable `codigo\_morse` que no está definida —dijo Mateo—. Pero está siendo llamada desde `transmitir\_mensaje`. Eso significa que el generador puede transmitir en código Morse. No solo en frecuencias.

—¿Y cómo nos ayuda eso?

—Porque si el sistema falla, revela información. Y en Python, podemos controlar esos fallos.

Try/Except: Atrapando errores como olas

```
# =====
# SISTEMA TOLERANTE A ERRORES
# =====

def conectar_generador(ip):
    """Intenta conectar con el generador submarino."""
    try:
        print(f" Conectando a {ip}...")
        # Simular conexión fallida
        if "192.168" in ip:
            raise ConnectionError("Red local no disponible")
        elif "10.0" in ip:
            raise TimeoutError("Tiempo de espera agotado")
        else:
            print(" Conexión exitosa")
            return True
    except ConnectionError as e:
        print(f" Error de conexión: {e}")
    return False
```

```

except TimeoutError as e:
    print(f"    Timeout: {e}")
    return False
except Exception as e:
    print(f"    Error inesperado: {e}")
    return False

conectar_generador("192.168.1.100")
conectar_generador("10.0.0.5")
conectar_generador("172.16.0.1")

```

Capturando múltiples excepciones

```

def leer_configuracion_generador(ruta_archivo):
    """Lee la configuración del generador desde un archivo."""
    try:
        with open(ruta_archivo, "r", encoding="utf-8") as f:
            config = f.read()

        try:
            niveles = [int(x) for x in config.split(",")]
            print(f"    Configuración cargada: {niveles}")
            return niveles
        except ValueError:
            print(f"    Formato inválido: '{config.strip()}' no es numérico")
            return []

    except FileNotFoundError:
        print(f"    Archivo no encontrado: {ruta_archivo}")
        return []
    except PermissionError:
        print(f"    Sin permisos para leer: {ruta_archivo}")
        return []

print("\n=== LEYENDO CONFIGURACIÓN ===\n")

# Crear archivos de prueba
with open("config_valida.txt", "w", encoding="utf-8") as f:
    f.write("12,15,9,21,5,14,20,15")
with open("config_invalida.txt", "w", encoding="utf-8") as f:
    f.write("doce,quince,nueve")

leer_configuracion_generador("config_valida.txt")
leer_configuracion_generador("config_invalida.txt")
leer_configuracion_generador("no_existe.txt")

```

Else y Finally

```

print("\n=== PROCESO CON GARANTÍAS ===\n")

def procesar_mensaje_ola(archivo_mensaje):
    """Procesa un mensaje codificado en olas."""

```

```

print(f"Iniciando procesamiento de: {archivo_mensaje}")

try:
    with open(archivo_mensaje, "r", encoding="utf-8") as f:
        datos = f.read()

        if not datos.strip():
            raise ValueError("Mensaje vacío")

        resultado = len(datos)

except FileNotFoundError:
    print(f" ERROR: Mensaje no encontrado")
    resultado = -1
except ValueError as e:
    print(f" ERROR: {e}")
    resultado = -1
else:
    print(f" Mensaje procesado: {resultado} caracteres")
    if "SOS" in datos or "AYUDA" in datos:
        print(f" ¡MENSAJE DE AUXILIO DETECTADO!")
finally:
    print(f" → Operación finalizada: {archivo_mensaje}\n")

return resultado

with open("mensaje_mar.txt", "w", encoding="utf-8") as f:
    f.write("SOS: AYUDA - OPERADOR SECUESTRADO")
procesar_mensaje_ola("mensaje_mar.txt")
procesar_mensaje_ola("no_existe.txt")

```

Excepciones personalizadas: Nudos propios

Mateo creó sus propias excepciones para el caso:

```

# =====
# EXCEPCIONES PERSONALIZADAS
# =====

class GeneradorDesconectadoError(Exception):
    """Error cuando el generador no responde."""
    pass

class OlaAnómalaNoDetectadaError(Exception):
    """Error cuando no se encuentra el patrón esperado."""
    def __init__(self, patron_esperado, patron_encontrado):
        super().__init__(f"Se esperaba '{patron_esperado}', se encontró '{patron_encontrado}'")

class EvidenciaInsuficienteError(Exception):
    """Error cuando no hay suficientes pruebas."""
    pass

```

```
# --- USANDO EXCEPCIONES PERSONALIZADAS ---

def verificar_generador(estado):
    """Verifica el estado del generador."""
    if estado == "desconectado":
        raise GeneradorDesconectadoError("El generador submarino no responde")
    elif estado == "anómalo":
        print("    Generador activo pero con comportamiento anómalo")
    else:
        print("    Generador funcionando normalmente")

def verificar_patron_ola(esperado, real):
    """Verifica que el patrón de ola coincida."""
    if esperado != real:
        raise OlaAnómalaNoDetectadaError(esperado, real)
    return True

print("\n=== VALIDACIONES DEL GENERADOR ===\n")

try:
    verificar_generador("desconectado")
except GeneradorDesconectadoError as e:
    print(f"    {e}")

try:
    verificar_patron_ola("--//0", "--//X")
except OlaAnómalaNoDetectadaError as e:
    print(f"    {e}")

print("\n    Validaciones completadas")
```

El error que reveló al operador

Mateo ejecutó una simulación del generador con datos reales. El programa falló con un error que mostraba el nombre del operador:

```
# --- SIMULACIÓN CON ERROR ---

try:
    # Intentar ejecutar el firmware original
    print("\n=== EJECUTANDO FIRMWARE ORIGINAL ===\n")

    usuarios_autorizados = ["carlos.parra", "miguel.soto", "luisa.rivas"]
    operador_actual = "carlos.parra"

    if operador_actual not in usuarios_autorizados:
        raise PermissionError(f"Operador '{operador_actual}' no autorizado")

    print(f" Acceso concedido a {operador_actual}")

    # Simular un comando
    try:
```

```

        raise NameError("variable 'firma_digital_LR2026' no definida")
    except NameError as e:
        print(f"    Error revelador: {e}")
        print(f"    → La variable 'firma_digital_LR2026' sugiere:")
        print(f"        • Iniciales: LR = ¿Luisa Rivas?")
        print(f"        • Año: 2026")
        print(f"        • Firma digital: posiblemente de la bióloga")

    except PermissionError as e:
        print(f"    {e}")

    print("\n → Análisis completo: el operador original podría ser Luisa Rivas")

```

—Luisa Rivas —dijo Mateo—. La bióloga marina. La que denunció el ruido submarino... Pero firmó el código del generador. ¿Ella lo programó o alguien usó su firma?

—O ella está infiltrada —dijo Rafa—. Trabajando desde adentro para exponer el proyecto.

—Hay una forma de saberlo. Necesito ver el código completo. El firmware está dividido en **módulos**. Y cada módulo tiene un autor.

Enigmas

Enigma 9.1: Conexión segura al generador

Escribe una función `conectar(ip, puerto)` que intente la conexión y capture `ConnectionError` mostrando "No se pudo conectar al generador".

Enigma 9.2: Lector de datos marinos tolerante

Pide al usuario un nombre de archivo. Intenta abrirlo. Si no existe, captura el error y crea el archivo con contenido por defecto.

Enigma 9.3: Excepción personalizada

Crea `OlaNoDetectadaError` y una función `detectar_ola(altura)` que la lance si la altura es 0.

Enigma 9.4: Finally para cerrar conexión

Simula una conexión al generador que use `finally` para mostrar "Cerrando conexión con el generador submarino" pase lo que pase.

Lo que aprendiste

- `try/except` captura errores sin romper el programa
- Se capturan tipos específicos (`ValueError`, `FileNotFoundError`, etc.)
- `else` se ejecuta si no hubo error
- `finally` se ejecuta siempre
 - Puedes crear **excepciones personalizadas**
- Los errores revelan información de la estructura interna

Mateo tenía un nuevo sospechoso: Luisa Rivas. Pero no estaba seguro de si era cómplice o víctima. La firma digital LR2026 podía haber sido plantada.

—Necesito entender la estructura completa del proyecto —dijo—. El firmware está organizado en **módulos**. Como las diferentes partes de un barco: motor, navegación, comunicaciones. Cada módulo es un archivo separado.

Capítulo 10: Los Módulos del Puerto

Conceptos: Módulos, paquetes, `import`, `\_\_name\_\_`

Mateo había logrado acceder al repositorio de código del proyecto LOCUMENTO. No era un solo archivo: era un **paquete** completo de Python, con múltiples módulos.

```
locuento/  
├─ __init__.py  
├─ generador.py      # Control del generador de olas  
├─ comunicaciones.py # Transmisión de mensajes  
├─ monitoreo.py     # Lectura de sensores  
├─ config.py         # Configuración del sistema  
└─ secreto.py        # ¿Módulo oculto?
```

—El módulo `secreto.py` no debería estar ahí —dijo Mateo—. No aparece en la documentación oficial.

Import: Conectando módulos como redes de pesca

```
# =====  
# EXPLORANDO LOS MÓDULOS DE LOCUMENTO  
# =====  
  
# Crear módulos simulados  
  
with open("locuento_generador.py", "w", encoding="utf-8") as f:  
    f.write('""Control del generador de olas."""')  
def activar(altura, frecuencia):  
    return f"Generando ola: {altura}m cada {frecuencia}s"  
  
def desactivar():  
    return "Generador detenido"  
  
VERSION = "2.0"  
''')  
  
with open("locuento_comunicaciones.py", "w", encoding="utf-8") as f:  
    f.write('""Transmisión de mensajes por olas."""')  
def transmitir(mensaje):  
    return f"Transmitiendo: {mensaje}"
```

```

def codificar(texto):
    return [ord(c) - 64 for c in texto.upper() if c.isalpha()]
'''

with open("locuento_secreto.py", "w", encoding="utf-8") as f:
    f.write('""""Módulo secreto - Solo personal autorizado.""""
__clave = "ANC0N2026"
def revelar_plan():
    return "Plan: Privatizar playas de Ancón mediante olas artificiales"
def verificar_clave(clave):
    return clave == __clave
'''

# Importar y usar
import locuento_generador as gen
import locuento_comunicaciones as com
import locuento_secreto as sec

print("=== MÓDULOS DEL PROYECTO LOCUENTO ===\n")

print(gen.activar(2.4, 8))
print(com.transmitir("PRUEBA"))
print(f"Versión del generador: {gen.VERSION}")

```

Import selectivo

```

from locuento_generador import activar, VERSION
print(f"\n{activar(1.8, 12)}")
print(f"Versión: {VERSION}")

```

El módulo secreto

```

print("\n=== MÓDULO SECRETO ===")
print(sec.revelar_plan())

clave = input("\nIngresa la clave de acceso: ")
if sec.verificar_clave(clave):
    print(" Acceso concedido al módulo secreto")
else:
    print(" Acceso denegado")

```

`if \_\_name\_\_ == "\_\_main\_\_"`: Puerto principal

```

# --- Crear módulo con ejecución condicional ---
with open("analisis_olas.py", "w", encoding="utf-8") as f:
    f.write('""""Módulo de análisis de olas.""""
def analizar(alturas):
    return {
        "promedio": sum(alturas) / len(alturas) if alturas else 0,
        "maxima": max(alturas) if alturas else 0,
    }

```

```

        "total": len(alturas)
    }

def main():
    print("=== ANALIZADOR DE OLAS ===\n")
    alturas = [float(x) for x in input("Alturas (separadas por coma): ").split(",")]
    resultado = analizar(alturas)
    for k, v in resultado.items():
        print(f" {k}: {v}")

if __name__ == "__main__":
    main()
'''

print("\n=== IMPORTANDO COMO MÓDULO ===")
import analisis_olas
resultado = analisis_olas.analizar([1.2, 1.8, 2.4, 1.5])
print(f"Promedio: {resultado['promedio']}")

```

Enigmas

Enigma 10.1: Crea un módulo de playas

Crea `playas\_peru.py` con funciones `lista\_playas()` y `mejor\_playa()`. Luego impórtalo.

Enigma 10.2: if __name__

Agrega a tu módulo un bloque que muestre "Módulo de playas - Modo autónomo" si se ejecuta directamente.

Enigma 10.3: Import selectivo

Del módulo `locuento\_generador`, importa solo `desactivar`. Úsalo.

Lo que aprendiste

- Los **módulos** son archivos `.py` reutilizables
- `import modulo` / `from modulo import funcion`
- `as` crea alias
- `\_\_name\_\_ == "\_\_main\_\_"` para ejecución directa

Mateo ejecutó el módulo secreto. Lo que encontró lo dejó helado: el plan no era solo privatizar Ancón. El proyecto LOCUENTO era una prueba para un sistema mayor que abarcaría toda la Costa Verde.

Pero alguien más estaba involucrado. Alguien que había dejado su firma en el código. Una firma en forma de **clase**.

Capítulo 11: La Clase del Surfista

Conceptos: POO, clases, objetos, atributos, métodos

Mateo había descargado todo el código del proyecto LOCUMENTO. Pero el código no solo tenía funciones: tenía **clases**. Alguien había modelado el proyecto como un sistema de objetos.

—El mar no es solo datos —dijo—. El mar es un objeto. Una ola es un objeto. Un surfista es un objeto. Todo en el mar puede representarse como una clase.

Clases: El molde del surfista

Una **clase** es un molde para crear objetos. Como un molde de tabla de surf: defines la forma, y luego creas tablas (objetos) a partir de ella.

```
# =====
# MODELANDO EL OCÉANO CON CLASES
# =====

class Ola:
    """Representa una ola del mar."""

    def __init__(self, altura, frecuencia, direccion):
        self.altura = altura
        self.frecuencia = frecuencia
        self.direccion = direccion
        self.rota = False

    def romper(self):
        """La ola rompe en la orilla."""
        self.rota = True
        return f" ¡Ola de {self.altura}m rompiendo!"

    def es_surfeable(self):
        return 0.8 <= self.altura <= 3.0

    def info(self):
        return f"Ola: {self.altura}m, cada {self.frecuencia}s, dirección {self.direccion}"

ola_1 = Ola(1.8, 12, "SO")
ola_2 = Ola(2.4, 8, "O")
```

```

ola_3 = Ola(0.5, 20, "NO")

print("=== OLAS CREADAS ===")
print(ola_1.info())
print(ola_2.info())
print(ola_3.info())

print(f"\n¿Ola 1 surfeable? {ola_1.es_surfeable()}")
print(f"¿Ola 3 surfeable? {ola_3.es_surfeable()}")

print(f"\n{ola_2.romper()}")

```

La clase Implicado

Mateo modeló a los implicados como objetos:

```

class Implicado:
    """Persona involucrada en el proyecto LOCUMENTO."""

    def __init__(self, nombre, rol, nivel_riesgo=5):
        self.nombre = nombre
        self.rol = rol
        self.nivel_riesgo = nivel_riesgo
        self.evidencias = []
        self.interrogado = False

    def agregar_evidencia(self, evidencia):
        self.evidencias.append(evidencia)
        self.nivel_riesgo += 1
        print(f"    Evidencia contra {self.nombre}: {evidencia}")

    def interrogar(self):
        self.interrogado = True
        print(f"\n    INTERROGANDO A {self.nombre.upper()}")
        print(f"        Rol: {self.rol}")
        print(f"        Riesgo: {self.nivel_riesgo}/10")
        print(f"        Evidencias: {len(self.evidencias)}")
        if self.nivel_riesgo >= 7:
            return f"        → {self.nombre} es ALTAMENTE SOSPECHOSO"
        elif self.nivel_riesgo >= 4:
            return f"        → {self.nombre} requiere vigilancia"
        return f"        → {self.nombre} tiene perfil bajo"

carlos = Implicado("Carlos Parra", "Ingeniero Naval", 8)
luisa = Implicado("Luisa Rivas", "Bióloga Marina", 3)
soto = Implicado("Miguel Ángel Soto", "Empresario", 9)

print("\n=== IMPLICADOS CREADOS ===")
carlos.agregar_evidencia("Diseñó el generador")
soto.agregar_evidencia("Financia el proyecto")
soto.agregar_evidencia("Dueño del terreno")
print(carlos.interrogar())
print(soto.interrogar())

```

Atributos de clase

```
class Investigacion:
    nombre_caso = "Operación LOCUMENTO - Contaminación de Olas"
    total_investigadores = 0

    def __init__(self, investigador):
        self.investigador = investigador
        self.implificados = []
        self.evidencias = []
        self.estado = "abierta"
        Investigacion.total_investigadores += 1

    def agregar_implificado(self, implificado):
        self.implificados.append(implificado)

    def resumen(self):
        print(f"\n=== {self.nombre_caso} ===")
        print(f"Investigador: {self.investigador}")
        print(f"Implificados: {len(self.implificados)}")
        print(f"Evidencias: {len(self.evidencias)}")
        print(f"Investigadores activos: {Investigacion.total_investigadores}")

caso = Investigacion("Mateo Sánchez")
caso.agregar_implificado(carlos)
caso.agregar_implificado(soto)
caso.evidencias.append("Firmware del generador")
caso.evidencias.append("Registro de activaciones")
caso.resumen()
```

Enigmas

Enigma 11.1: Clase TablaDeSurf

Crea una clase `TablaDeSurf` con: `marca`, `longitud`, `material`. Método `remar()` que muestre "Remando en [marca]".

Enigma 11.2: Clase con contador

Modifica `Ola` para que cuente cuántas olas se han creado en total.

Enigma 11.3: Sistema de playas

Crea una clase `Playa` con atributos `nombre`, `ubicacion`, `tiene\_muelle`. Método `info()`. Crea 3 playas.

Lo que aprendiste

- Una **clase** define un molde para objetos
- `\_\_init\_\_()` es el constructor
- `self` se refiere al propio objeto

- **Atributos de clase** son compartidos
- Los **métodos** son funciones del objeto

Mateo había modelado a los implicados como objetos. Pero notó que algunos compartían características: Soto y Parra tenían acceso al generador, ambos tenían altos niveles de riesgo. En cambio, Luisa Rivas era un caso distinto.

—Los implicados no son todos iguales —dijo Mateo—. Hay una jerarquía. Algunos heredan características de otros. Como las clases en Python.

Estaba a punto de descubrir la **herencia**.

Capítulo 12: La Herencia del Mar

Conceptos: Herencia, polimorfismo, encapsulación, `super()`

Mateo descubrió que el código del generador usaba herencia de clases. Había una clase base `Dispositivo`, y de ella heredaban `GeneradorOlas`, `SensorMarino`, y `TransmisorMensajes`.

—Es como el océano —dijo—. El `Mar` es la clase base. De él heredan `Playa`, `Ola`, `Corriente`. Todos comparten atributos del mar, pero cada uno tiene sus propios métodos.

Herencia: La familia del océano

```
# =====
# HERENCIA EN EL OCÉANO DIGITAL
# =====

class Embarcacion:
    """Clase base para todas las embarcaciones."""

    def __init__(self, nombre, eslora, capacidad):
        self.nombre = nombre
        self.eslora = eslora # metros
        self.capacidad = capacidad # personas
        self._activo = True

    def zarpar(self):
        return f" {self.nombre} ha zarpado"

    def atracar(self):
        return f" {self.nombre} ha atracado"

    def info(self):
        return f"{self.nombre}: {self.eslora}m, {self.capacidad} pers."

class BarcoPesquero(Embarcacion):
    def __init__(self, nombre, eslora, capacidad, tipo_red):
        super().__init__(nombre, eslora, capacidad)
        self.tipo_red = tipo_red
        self.pescado = 0

    def pescar(self):
```



```

        self.pescado += 100
        return f" {self.nombre} pescó 100 kg con red {self.tipo_red}"

    def info(self):
        return f"{super().info()} | Pescador, red: {self.tipo_red}"

class YatePrivado(Embarcacion):
    def __init__(self, nombre, eslora, capacidad, dueno):
        super().__init__(nombre, eslora, capacidad)
        self.dueno = dueno

    def info(self):
        return f"{super().info()} | Yate de {self.dueno}"

print("=== EMBARCACIONES DE ANCÓN ===\n")
don_eulogio = BarcoPesquero("Don Eulogio", 8, 4, "enmalle")
costa_azul = YatePrivado("Costa Azul", 25, 12, "Miguel Ángel Soto")

print(don_eulogio.info())
print(costa_azul.info())
print(don_eulogio.zarpar())
print(don_eulogio.pescar())

```

Polimorfismo: Mismo mensaje, diferente respuesta

```

print("\n=== POLIMORFISMO: TODOS ZARPAN ===\n")
barcos = [
    BarcoPesquero("Mares del Sur", 12, 6, "arrastre"),
    YatePrivado("Costa Azul", 25, 12, "Soto"),
    BarcoPesquero("Ancón I", 7, 3, "cerco"),
]
for barco in barcos:
    print(barco.info())
    print(f" → {barco.zarpar()}")

```

Herencia múltiple

```

class Capitan:
    def __init__(self, nombre, licencia):
        self.nombre_capitan = nombre
        self.licencia = licencia
    def dirigir(self):
        return f"Capitán {self.nombre_capitan} dirigiendo"

class Navegante:
    def __init__(self, sistema_gps):
        self.sistema_gps = sistema_gps
    def navegar(self, destino):
        return f"Navegando a {destino} con {self.sistema_gps}"

class BarcoModerno(Embarcacion, Capitan, Navegante):
    def __init__(self, nombre, eslora, capacidad, capitan, licencia, gps):

```

```

Embarcacion.__init__(self, nombre, eslora, capacidad)
Capitan.__init__(self, capitan, licencia)
Navegante.__init__(self, gps)

print("\n=== BARCO MODERNO ===")
b = BarcoModerno("Investigador I", 20, 15, "Mateo", "CAP-001", "GPS Pro")
print(b.info())
print(b.dirigir())
print(b.navegar("Generador LOCUMENTO"))

```

Encapsulación: Secretos del mar

```

class ProyectoSecreto:
    def __init__(self, nombre, presupuesto):
        self.nombre = nombre
        self._presupuesto = presupuesto
        self.__clave_acceso = "SOLO_INVESTIGACION"

    def obtener_presupuesto(self, nivel_acceso):
        if nivel_acceso >= 5:
            return f"Presupuesto: ${self._presupuesto:}"
        return " Acceso denegado"

p = ProyectoSecreto("LOCUMENTO", 2500000)
print(f"\nNombre: {p.nombre}")
print(p.obtener_presupuesto(3))
print(p.obtener_presupuesto(5))

```

Enigmas

Enigma 12.1: Jerarquía de playas

Crea `ZonaCostera`, `PlayaPublica` y `PlayaPrivada`. Cada una agrega un atributo.

Enigma 12.2: Polimorfismo con sonidos

Crea `SonidoMarino`, `Ola` hereda con `sonido()` "¡Swish!", `Barco` "¡Buuu!".

Enigma 12.3: Encapsulación bancaria

Clase `CuentaMaritima` con `\_\_saldo` privado. Métodos `depositar()` y `retirar()`.

Lo que aprendiste

- **Herencia:** `class Hijo(Padre):`
- `super().\_\_init\_\_()` llama al constructor padre
- **Polimorfismo:** mismos métodos, diferentes comportamientos
- **Herencia múltiple:** varias clases padre
- **Encapsulación:** `\_` y `\_\_` protegen atributos

Mateo tenía todas las piezas del rompecabezas. Clases, herencia, encapsulación. Pero eran

conceptos sueltos. Necesitaba unirlos en un solo programa que analizara las olas, detectara las artificiales, descifrara los mensajes y expusiera a los culpables.

Necesitaba un **proyecto integrador**.

Capítulo 13: El Proyecto Integrador

Conceptos: Proyecto completo — Analizador de Olas LOCUENTO

Mateo había reunido todas las piezas. Tenía el firmware del generador, los registros de activación, los mensajes codificados en las olas, y los nombres de los implicados. Ahora necesitaba construir una sola herramienta que unificara todo: un **analizador de olas** que descifrara el mensaje completo del proyecto LOCUENTO.

—Ya no es solo investigar —dijo—. Es construir. Voy a crear un programa que analice olas, detecte las artificiales, descifre los mensajes, y exponga el plan.

El Proyecto Integrador

El programa final integra todos los conceptos del libro:

- **Variables/tipos** para almacenar datos de olas
- **Strings/slicing** para descifrar mensajes
- **Listas/dicts** para organizar registros
- **Condicionales/bucles** para filtrar datos
- **Funciones/módulos** para organizar el código
- **Archivos/errores** para persistencia
- **POO/herencia** para modelar el sistema

```
# =====  
# PROYECTO INTEGRADOR: ANALIZADOR DE OLAS  
# =====  
# Detecta olas artificiales y descifra mensajes  
# del proyecto LOCUENTO  
  
import json  
from datetime import datetime  
  
# --- MÓDULOS ---  
  
class Ola:  
    """Representa una ola registrada."""
```

```

total_olas = 0

def __init__(self, altura, frecuencia, direccion, timestamp=None):
    self.altura = altura
    self.frecuencia = frecuencia
    self.direccion = direccion
    self.timestamp = timestamp or datetime.now().isoformat()
    self._es_artificial = False
    Ola.total_olas += 1

def es_surfeable(self):
    return 0.8 <= self.altura <= 3.0

def marcar_artificial(self):
    self._es_artificial = True

def es_artificial(self):
    return self._es_artificial

def info(self):
    return (f'Ola #{Ola.total_olas}: {self.altura}m, '
            f'{self.frecuencia}s, {self.direccion} | '
            f'{' Artificial' if self._es_artificial else ' Natural'}')

class OlaArtificial(Ola):
    def __init__(self, altura, frecuencia, direccion, mensaje_codificado):
        super().__init__(altura, frecuencia, direccion)
        self.mensaje_codificado = mensaje_codificado
        self.marcar_artificial()

    def descifrar_mensaje(self):
        return ''.join(chr(c + 64) for c in self.mensaje_codificado if 1 <= c <= 26)

# --- FUNCIONES ---

def filtrar_por_altura(olas, min_alt, max_alt):
    return [o for o in olas if min_alt <= o.altura <= max_alt]

def detectar_artificiales(olas):
    """Detecta olas artificiales por patrón de frecuencia exacta."""
    paises = {"15", "18", "20", "25", "30"} # frecuencias non-naturales
    return [o for o in olas if str(o.frecuencia) in paises]

def registrar_ola(archivo, ola):
    with open(archivo, "a", encoding="utf-8") as f:
        json.dump({
            "altura": ola.altura,
            "frecuencia": ola.frecuencia,
            "direccion": ola.direccion,
            "timestamp": ola.timestamp,
            "artificial": ola.es_artificial()

```

```

    }, f)
    f.write("\n")

def cargar_registros(archivo):
    try:
        with open(archivo, "r", encoding="utf-8") as f:
            return [json.loads(linea) for linea in f if linea.strip()]
    except FileNotFoundError:
        print(f" Archivo {archivo} no encontrado. Creando nuevo.")
        return []

def main():
    print("=" * 60)
    print("  ANALIZADOR DE OLAS - PROYECTO LOCUENTO")
    print("=" * 60)

    # Datos de ejemplo - registros del puerto
    registros_raw = [
        (1.2, 12, "SO", None), (2.4, 15, "O", [1, 14, 3, 15, 14]),
        (0.8, 8, "NO", None), (1.8, 18, "SO", [16, 18, 9, 22, 1, 20, 15]),
        (2.0, 20, "O", [3, 15, 19, 20, 1, 26, 21, 12]),
        (0.5, 25, "NO", [16, 12, 1, 14]), (1.5, 10, "SO", None),
        (2.2, 30, "O", [15, 12, 1, 19, 4, 5, 3, 15, 4, 9, 6, 9, 3, 1, 4, 15])
    ]

    olas = []
    for alt, freq, dir, msg in registros_raw:
        if msg:
            ola = OlaArtificial(alt, freq, dir, msg)
        else:
            ola = Ola(alt, freq, dir)
        olas.append(ola)

    print(f"\n Total olas registradas: {len(olas)}")

    artificiales = [o for o in olas if o.es_artificial()]
    print(f" Olas artificiales detectadas: {len(artificiales)}")

    print("\n=== MENSAJES DESCIFRADOS ===")
    mensaje_completo = []
    for ola in artificiales:
        if ola.mensaje_codificado:
            msg = ola.descifrar_mensaje()
            print(f"  {msg}")
            mensaje_completo.append(msg)

    print(f"\n Mensaje completo: {' '.join(mensaje_completo)}")

    print(f"\n Olas surfeables: {len([o for o in olas if o.es_surfeable()])}")

    # Persistencia
    archivo = "registros_olas.json"

```

```

for ola in olas:
    registrar_ola(archivo, ola)
print(f"\n Registros guardados en {archivo}")

print("\n" + "=" * 60)
print("  INVESTIGACIÓN COMPLETA")
print("=" * 60)
print("  El proyecto LOCUMENTO usaba olas artificiales")
print("  para transmitir mensajes cifrados y manipular")
print("  las condiciones del mar en Ancón.")
print("=" * 60)

if __name__ == "__main__":
    main()

```

Enigmas

Enigma 13.1: Ampliar el analizador

Agrega una función `generar\_reporte()` que cree un archivo `.txt` con el resumen.

Enigma 13.2: Nuevo tipo de ola

Crea `OlaTormenta` que herede de `Ola` y tenga atributo `intensidad`.

Enigma 13.3: Mapa de calor

Crea una función que muestre un mapa de calor de las olas por hora usando `\*` (asteriscos).

Lo que aprendiste

- Cómo integrar múltiples conceptos en un solo proyecto
- Cómo modelar un sistema real con POO
- Cómo persistir datos y cargarlos
- Cómo estructurar un programa completo

Mateo tenía el analizador funcionando. Pero aún faltaba lo más importante: presentar las evidencias ante todos los implicados. El momento de la verdad se acercaba. Y él estaría listo.

Capítulo 14: Revelación en el Muelle

Conceptos: Repaso final — Poniendo todo junto

La noche era clara en Ancón. Mateo estaba en el muelle, su laptop abierta mostrando el analizador de olas. Frente a él, el mar estaba sospechosamente tranquilo. Demasiado tranquilo.

Había citado a todos los implicados. Soto, Parra, Luisa. Y, para su sorpresa, el viejo Eulogio.

—Ustedes se preguntan por qué los reuní aquí —dijo Mateo—. Porque el mar habla. Y yo aprendí a escucharlo... con Python.

El momento de la verdad

Mateo mostró el analizador en vivo. Cada ola artificial quedaba marcada con sus mensajes descifrados.

—El generador está ahí —señaló hacia el horizonte—. A 500 metros de la costa. Transmite datos en las olas. ¿Saben cómo lo sé? Porque las olas naturales no tienen frecuencias de 15, 18, 20, 25 y 30 segundos. Esas frecuencias son mensajes. Son coordenadas.

```
# === REPASO FINAL: EL CÓDIGO DE LA VERDAD ===

mensajes_olas = {
    "15s": "ANCÓN",
    "18s": "PRIVATO",
    "20s": "COSTAZUL",
    "25s": "PLAN",
    "30s": "OLASDECODIFICADO"
}

print("=== DECODIFICACIÓN COMPLETA ===")
for freq, msg in mensajes_olas.items():
    print(f" Frecuencia {freq} → {msg}")

mensaje = " ".join(mensajes_olas.values())
print(f"\n Mensaje completo: {mensaje}")

# La evidencia final
evidencias = [
    "Firmware del generador (módulo secreto)",
    "Registros de activación (30 días)",
    "Transferencias de Costa Azul S.A.",
    "Mensajes descifrados en 9 olas artificiales",
```



```

    "Testimonio del capitán del Don Eulogio",
]

print("\n=== EVIDENCIAS RECOPIADAS ===")
for i, ev in enumerate(evidencias, 1):
    print(f" {i}. {ev}")

# Búsqueda de cómplices
implicados = {
    "Soto (Costa Azul)": {
        "rol": "Financiamiento",
        "evidencias": 4,
        "nivel": "Alto"
    },
    "Parra (Ingeniero)": {
        "rol": "Construcción del generador",
        "evidencias": 3,
        "nivel": "Alto"
    },
    "Rivas (Bióloga)": {
        "rol": "Informes ambientales falsos",
        "evidencias": 2,
        "nivel": "Medio"
    },
}

print("\n=== NIVEL DE IMPLICACIÓN ===")
for nombre, datos in implicados.items():
    print(f" {nombre}:")
    print(f" Rol: {datos['rol']}")
    print(f" Evidencias: {datos['evidencias']}")
    print(f" Nivel: {datos['nivel']}")

```

La confesión

Soto se levantó, pálido.

—Está bien. Es cierto. Usamos el generador para crear olas artificiales. Pero no para privatizar Ancón. Bueno... también para eso. Pero el verdadero propósito era probar un sistema de comunicación submarina. La Marina estuvo involucrada al principio.

—¿La Marina? —preguntó Luisa.

—El proyecto LOCUMENTO comenzó como un proyecto de defensa. Comunicación por olas. Invisible, indetectable. Pero cuando lo cancelaron, decidí seguir adelante por mi cuenta. Para la inmobiliaria.

—Y para eso —dijo Mateo— manipularon las olas, alejaron a los surfistas, y planearon comprar toda la costa.

Don Eulogio se levantó lentamente.

—Joven Mateo —dijo—. Yo sabía todo esto desde el principio. El generador está en mi lancha. Yo ayudé a Parra a instalarlo.

Todos miraron al viejo pescador.

—Pero no para Soto —continuó—. Para la comunidad. Grabé todo. Tengo meses de evidencias.

Esperaba a alguien como usted para exponerlo.

El viejo pescador sacó un USB de su bolsillo.

—Aquí está todo: el firmware original, los planos, las transferencias. Mi nieta me enseñó a usar Python. La evidencia... la guardé en un diccionario.

```
# El USB de Don Eulogio
evidencia_final = {
    "firmware": "locuento_v2_original.bin",
    "planos": ["generador.dwg", "sistema_comunicacion.dwg"],
    "transferencias": [
        {"de": "Costa Azul S.A.", "a": "Parra Ingeniería", "monto": 150000},
        {"de": "Costa Azul S.A.", "a": "Soto Holding", "monto": 500000},
        {"de": "Soto Holding", "a": "Municipalidad de Ancón", "monto": 20000},
    ],
    "testigos": ["Eulogio Quispe", "Ana María Huerta", "Pedro Castillo"],
    "fecha_inicio": "2025-03-15",
}

print("=== EVIDENCIA FINAL (USB DON EULOGIO) ===")
for clave, valor in evidencia_final.items():
    print(f" {clave}: {valor}")
```

Epílogo del capítulo

La noticia estalló en todos los medios. "Costa Azul S.A. investigada por manipulación ambiental". Soto fue detenido. Parra también. Luisa fue absuelta por colaborar con la investigación.

El generador fue desmantelado. La Municipalidad de Ancón declaró la zona como "Reserva de Olas" —la primera en el Perú— protegiendo el derecho de los surfistas a las olas naturales.

Mateo volvió a surfear al día siguiente. Las olas eran perfectas. Naturales. Libres.

Pero antes de guardar su tabla, miró el horizonte. En la pantalla de su laptop, en la orilla, un nuevo mensaje parpadeaba:

```
SISTEMA_OLAS: NUEVO GENERADOR DETECTADO - CALLAO - FRECUENCIA 22
```

El código de las olas nunca dejaba de hablar.

Enigmas

Enigma 14.1: Sistema de alertas

Crea una función que revise un archivo de registros y detecte frecuencias sospechosas.

Enigma 14.2: Base de datos de playas

Usando un archivo JSON, crea un sistema que almacene y consulte playas protegidas vs amenazadas.

Enigma 14.3: Cifrado propio

Crea tu propio sistema de cifrado usando olas como metáfora. Cada letra se convierte en una frecuencia.

Lo que aprendiste

- Todos los conceptos de Python pueden trabajar juntos
 - La evidencia digital es poderosa
 - El código puede exponer la verdad
 - La comunidad puede usar la tecnología para protegerse
-

Capítulo 15: Epílogo — Ola Libre

Conceptos: Reflexión final — Python como herramienta de cambio

Pasaron tres meses.

Ancón había cambiado. La "Reserva de Olas" era ahora un modelo para otras caletas del Perú. La Municipalidad había instalado sensores —manejados por la comunidad— para monitorear la calidad de las olas y detectar cualquier intento de manipulación.

Mateo abrió su laptop una mañana, frente al mar. El sol salía sobre el puerto. Las olas rompían con un ritmo perfecto.

—Hora de la lectura de olas —dijo.

```
# =====
# EPÍLOGO: OLA LIBRE - MONITOREO COMUNITARIO
# =====

from datetime import datetime
import json

class MonitorOlas:
    """Sistema de monitoreo comunitario de Ancón."""

    def __init__(self, archivo="monitoreo_ancon.json"):
        self.archivo = archivo
        self.lecturas = []
        self.cargar_historial()

    def cargar_historial(self):
        try:
            with open(self.archivo, "r", encoding="utf-8") as f:
                self.lecturas = [json.loads(l) for l in f if l.strip()]
            print(f" Historial cargado: {len(self.lecturas)} registros")
        except FileNotFoundError:
            print(" Nuevo archivo de monitoreo")

    def registrar_ola(self, altura, frecuencia, direccion):
        lectura = {
            "fecha": datetime.now().isoformat(),
            "altura": altura,
```

```

        "frecuencia": frecuencia,
        "direccion": direccion,
        "natural": not self._es_sospechosa(frecuencia)
    }
    self.lecturas.append(lectura)
    with open(self.archivo, "a", encoding="utf-8") as f:
        json.dump(lectura, f)
        f.write("\n")

    estado = " Natural" if lectura["natural"] else " ¡Sospechosa!"
    print(f"{estado} | {altura}m | {frecuencia}s | {direccion}")

def _es_sospechosa(self, frecuencia):
    return frecuencia in (15, 18, 20, 25, 30)

def reporte(self):
    total = len(self.lecturas)
    sospechosas = [l for l in self.lecturas if not l["natural"]]
    print(f"\n=== REPORTE DE MONITOREO ===")
    print(f"Total registros: {total}")
    print(f"Olas sospechosas: {len(sospechosas)}")
    print(f"Playas protegidas: Ancón, Tuquillo, Hermosa")
    print(f"Estado: Libre" if len(sospechosas) == 0 else f" Alerta")
    return total

# Iniciar monitoreo del día
monitor = MonitorOlas()

print("\n=== LECTURA MATINAL DE OLAS ===")
print("Ancón, 06:00 AM\n")

monitor.registrar_ola(1.5, 10, "SO")
monitor.registrar_ola(0.8, 12, "O")
monitor.registrar_ola(2.0, 8, "NO")
monitor.registrar_ola(1.2, 20, "SO") # ¡Frecuencia sospechosa!

print(f"\nTotal olas hoy: {len(monitor.lecturas)}")

```

Mateo sonrió. La última lectura mostraba una frecuencia de 20 —la misma que usaba el generador. Pero esta vez no era el generador. Era el mar, recuperando su ritmo natural. O tal vez... alguien probando los límites del sistema.

—El código de las olas sigue vivo —dijo—. Pero ahora todos pueden leerlo.

Cerró la laptop, agarró su tabla, y corrió hacia el mar. La primera ola lo recibió como un viejo amigo.

Python es tu tabla de surf

En este libro aprendiste que Python no es solo un lenguaje de programación. Es una herramienta para entender el mundo, para resolver misterios, para proteger lo que amas.

Así como una tabla de surf te permite cabalgar las olas, Python te permite cabalgar los datos. Así como el surfista lee el mar, el programador lee el código. Así como una ola puede esconder un

mensaje, un programa puede revelar una verdad.

Sigue aprendiendo. Sigue preguntando. Y recuerda:

> **El océano tiene su propio código. Y ahora tú sabes cómo leerlo.**

Proyecto Final (Opcional)

Construye tu propio sistema de monitoreo

Usando todo lo aprendido, crea un sistema que:

1. **Registre** olas desde un archivo o entrada del usuario
2. **Analice** patrones sospechosos (frecuencias, alturas anómalas)
3. **Descifre** mensajes ocultos en las olas
4. **Almacene** los datos en archivos JSON
5. **Genere** reportes en texto
6. **Use clases** para modelar olas, playas, y monitoreo
7. **Maneje errores** con try/except

```
# ESQUELETO DEL PROYECTO FINAL
class SistemaMonitoreoOlas:
    def __init__(self):
        self.olas = []
        self.playas = []

    def agregar_ola(self, ola):
        pass # Implementa

    def analizar_patrones(self):
        pass # Implementa

    def generar_reporte(self):
        pass # Implementa

    def main(self):
        pass # Implementa

if __name__ == "__main__":
    sistema = SistemaMonitoreoOlas()
    sistema.main()
```

¡Felicitaciones! Has completado **Código de Olas: El Misterio del Puerto de Ancón**.
Ahora eres un programador Python... y un lector del mar.

Código fuente

Todo el código de este libro (y los demás) está disponible gratuitamente en:

[github.com/SistGoyPeru/Libros](https://github.com/SistGoyPeru/Libros)

¿Comentarios, sugerencias, o encontraste un error? Escíbeme a:

alexgoyzueta2018@gmail.com

Conclusión

Lo que aprendiste en este libro

Has recorrido las olas de Ancón junto a Mateo Sánchez, desde el Capítulo 1 hasta el epílogo. Hoy has completado un proyecto que integra cada concepto de Python:

Concepto	Capítulo	Aplicación en el caso
Variables y tipos	Cap 1	Datos de olas y sospechosos
Strings y slicing	Cap 2	Descifrado de mensajes en el mar
Listas y tuplas	Cap 3	Organización de implicados
Diccionarios y sets	Cap 4	Base de datos del proyecto LOCUMENTO
Condicionales	Cap 5	Detección de olas artificiales
Bucles	Cap 6	Recorrido de registros del puerto
Funciones	Cap 7	Herramientas de análisis marino
Archivos	Cap 8	Registros de activación del generador
Manejo de errores	Cap 9	Sistema tolerante en el mar
Módulos y paquetes	Cap 10	Estructura del proyecto LOCUMENTO
POO	Cap 11	Modelado de olas e implicados
Herencia	Cap 12	Jerarquía de embarcaciones y dispositivos
Proyecto integrador	Cap 13-14-15	Analizador de Olas LOCUMENTO

¿Qué sigue?

1. **Mejora el analizador de olas.** Agrega visualización con ``matplotlib`` o conviértelo en una app web con ``flask``.
2. **Crea tu propio misterio costero.** Inventa una historia en otra caleta del Perú y modela los datos en Python.
3. **Explora más allá:** ``pandas`` para analizar oleajes históricos, ``requests`` para consumir datos de mareas en tiempo real, o ``raspberrypi`` para conectar sensores reales.
4. **Comparte tu conocimiento.** La mejor forma de aprender es enseñar. Escribe tu propia historia interactiva.

El mensaje final

El mar tiene su propio código. Las olas hablan en frecuencias, alturas y direcciones. Python te enseñó a leer ese código, a encontrar patrones donde otros solo ven agua, y a usar la tecnología para proteger lo que amas.

Mateo Sánchez lo descubrió. Don Eulogio lo sabía desde el principio. Y tú, lector, lo has vivido.

Ancón ya no es solo un puerto. Es un símbolo de cómo la comunidad, el conocimiento y el código

pueden defender la naturaleza.

"El mar nunca deja de hablar. Solo necesitas saber escucharlo."

Alex Goyzueta Delgado

Lima, 2026

Apéndice: Soluciones a los Enigmas

Capítulo 1

Enigma 1.1:

```
nombre_playa = "Ancón Norte"
temperatura_agua = 19.2
altura_ola = 2.1
es_segura = True
hora_medicion = "07:30"

print(f"=== PARTE DE OLAS ===")
print(f"Playa: {nombre_playa}")
print(f"Temperatura: {temperatura_agua}°C")
print(f"Altura: {altura_ola}m")
print(f"Segura: {es_segura}")
print(f"Hora: {hora_medicion}")
```

Enigma 1.2:

```
alturas = [1.5, 2.0, 1.8, 2.2, 1.9]
promedio = sum(alturas) / len(alturas)
print(f"Promedio de alturas: {promedio:.2f}m")
```

Enigma 1.3:

```
ola_binaria = "1.8:0.5:2.1"
partes = ola_binaria.split(":")
print(f"Altura: {partes[0]}m")
print(f"Frecuencia: {partes[1]}s")
print(f"Velocidad: {partes[2]}m/s")
```

Capítulo 2

Enigma 2.1:

```
frecuencia = "03-15-12-01-19"
numeros = frecuencia.split("-")
palabra = ""
for num in numeros:
    letra = chr(int(num) + 64)
    palabra += letra
    print(f"{num} → '{letra}'")
print(f"\nPalabra: {palabra}") # COLAS
```

Enigma 2.2:

```
texto = "B/T-MARES-DE-ANCÓN-2026"
pos = texto.find("ANCÓN")
extraido = texto[pos:pos+5]
print(extraido) # ANCÓN
```

Enigma 2.3:

```
mensaje = "ODATNEMIRP SE ODAGUA"
inverso = mensaje[::-1]
print(inverso) # "AGUA DE PRIMERMUNDO"... ¡"AGUA DE PRIMER MUNDO"!
```

Enigma 2.4:

```
sensor = " OLA: 2.4m - FREC: 12s "
limpio = sensor.strip()
sin_etiquetas = limpio.replace("OLA: ", "").replace("- FREC: ", "")
valores = sin_etiquetas.split()
altura = valores[0].replace("m", "")
frecuencia = valores[1].replace("s", "")
print(f"Altura: {altura}m")
print(f"Frecuencia: {frecuencia}s")
```

Capítulo 3

Enigma 3.1:

```
playas_de_lima = ["Ancón", "Miraflores", "Barranco", "La Punta", "Costa Verde"]
playas_de_lima.append("Santa María")
playas_de_lima.sort()
print(f"Total: {len(playas_de_lima)}")
print(f"Primera: {playas_de_lima[0]}")
print(f"Última: {playas_de_lima[-1]}")
```

Enigma 3.2:

```
accesos = [
    ["Ancón", True, True, "Estacionamiento"],
    ["Miraflores", True, False, "Parque"],
    ["Barranco", True, True, "Escaleras"],
]
print("Playas con acceso para discapacitados:")
for playa in accesos:
    if playa[2]:
        print(f" {playa[0]}")
```

Enigma 3.3:

```
coordenadas = [
    (-11.7739, -77.1529),
    (-11.7758, -77.1897),
    (-12.1200, -77.0300),
]
```

```
for lat, lon in coordenadas:
    print(f"Playa en latitud {lat}, longitud {lon}")
```

Capítulo 4

Enigma 4.1:

```
mi_playa = {
    "nombre": "Ancón",
    "ubicacion": "Lima, Perú",
    "temperatura_agua": 18.5,
    "tiene_estacionamiento": True,
    "ola_favorita": "La Derecha del Muelle"
}
for clave, valor in mi_playa.items():
    print(f"{clave}: {valor}")
```

Enigma 4.2:

```
implicados = {
    "Carlos Parra": {"nivel_sospecha": 8, "evidencias": ["Diseñó el generador", "Recibió transferencia"]},
    "Miguel Soto": {"nivel_sospecha": 9, "evidencias": ["Financia proyecto"]},
    "Luisa Rivas": {"nivel_sospecha": 3, "evidencias": ["Denunció ruido"]},
}
print("Nombres:", list(implicados.keys()))
print("\nSospecha > 5:")
for n, d in implicados.items():
    if d["nivel_sospecha"] > 5:
        print(f"  {n}: {d['nivel_sospecha']/10}")
print(f"\nEvidencias de Carlos Parra: {implicados['Carlos Parra']['evidencias']}")
```

Enigma 4.3:

```
surfistas = {"Mateo", "Rafa", "Lucía"}
buceadores = {"Rafa", "Pedro", "Lucía"}
kayakistas = {"Mateo", "Pedro", "Sofía"}

print(f"Surf y buceo: {surfistas & buceadores}")
print(f"Solo surf: {surfistas - buceadores - kayakistas}")
print(f"Todos: {surfistas | buceadores | kayakistas}")
```

Capítulo 5

Enigma 5.1:

```
altura = float(input("Altura de la ola (m): "))
if altura < 1.0:
    print("Ola pequeña")
elif altura <= 2.5:
    print("Ola ideal para surf")
else:
    print("Ola peligrosa")
```

Enigma 5.2:

```

implicados = {
    "Carlos Parra": {"nivel_sospecha": 8, "involucrado": True, "evidencias": ["diseñó generador"]},
    "Miguel Soto": {"nivel_sospecha": 9, "involucrado": True, "evidencias": ["financia"]},
    "Luisa Rivas": {"nivel_sospecha": 3, "involucrado": False, "evidencias": []},
    "Capitán Paredes": {"nivel_sospecha": 5, "involucrado": False, "evidencias": ["no investigó"]},
}

print("Sospecha >= 7:")
for n, d in implicados.items():
    if d["nivel_sospecha"] >= 7:
        print(f" {n}")

print("\nInvolucrados Y con evidencias:")
for n, d in implicados.items():
    if d["involucrado"] and d["evidencias"]:
        print(f" {n}")

print("\nNo involucrados PERO sospecha > 4:")
for n, d in implicados.items():
    if not d["involucrado"] and d["nivel_sospecha"] > 4:
        print(f" {n}")

```

Enigma 5.3:

```

temp = float(input("Temperatura del agua (°C): "))
if temp < 16:
    print("Agua muy fría, usa traje de neopreno")
elif 16 <= temp <= 20:
    print("Agua templada, buenas condiciones")
else:
    print("Agua cálida, ideal para todo el día")

```

Capítulo 6

Enigma 6.1:

```

playas = ["Ancón", "Miraflores", "Barranco", "Costa Verde"]
for i, playa in enumerate(playas, 1):
    print(f"{i}. {playa}")

```

Enigma 6.2:

```

distancia = 0
while distancia < 100:
    distancia += 10
    print(f" Avanzando... {distancia}m")
print("¡GENERADOR ENCONTRADO!")

```

Enigma 6.3:

```

temperaturas = [18.2, 19.0, -5, 18.5, 500, 19.1]
for t in temperaturas:
    if t < 0 or t > 50:
        continue

```

```
print(f"Temperatura válida: {t}°C")
```

Enigma 6.4:

```
for i in range(1, 5):  
    print("*" * i)
```

Capítulo 7

Enigma 7.1:

```
def recomendar_traje(temp):  
    if temp < 16:  
        return "Traje de neopreno 5/4mm"  
    elif temp <= 20:  
        return "Traje corto"  
    else:  
        return "Solo shorts"  
  
print(recomendar_traje(15))  
print(recomendar_traje(18))  
print(recomendar_traje(22))
```

Enigma 7.2:

```
def distancia_buceo(minutos, ritmo=10):  
    return minutos * ritmo  
  
print(f"Distancia: {distancia_buceo(30)}m") # 300m  
print(f"Distancia: {distancia_buceo(30, 15)}m") # 450m
```

Enigma 7.3:

```
def alerta_ola(altura=1.5, peligro=False):  
    if peligro:  
        print(f" ¡PELIGRO! Ola de {altura}m detectada")  
    else:  
        print(f"Ola de {altura}m - condiciones normales")  
  
alerta_ola()  
alerta_ola(2.8, True)
```

Enigma 7.4:

```
def info_playa(nombre, tiene_muelle, tiene_estacionamiento):  
    puntuacion = sum([tiene_muelle, tiene_estacionamiento])  
    rec = "Excelente" if puntuacion == 2 else "Regular" if puntuacion == 1 else "Básica"  
    return puntuacion, rec  
  
puntos, rec = info_playa("Ancón", True, True)  
print(f"Puntuación: {puntos}/2 - {rec}")
```

Capítulo 8

Enigma 8.1:

```
with open("mi_diario_mar.txt", "w", encoding="utf-8") as f:
    f.write("Fecha: 27-06-2026\n")
    f.write("Hoy aprendí a leer y escribir archivos en Python.\n")
    f.write("El mar de Ancón guarda secretos en sus olas.\n")

with open("mi_diario_mar.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

Enigma 8.2:

```
with open("temperaturas_mar.txt", "w", encoding="utf-8") as f:
    f.write("18.5\n19.0\n17.8\n18.2\n19.5\n")

with open("temperaturas_mar.txt", "r", encoding="utf-8") as f:
    temps = [float(linea.strip()) for linea in f]

promedio = sum(temps) / len(temps)
print(f"Temperaturas: {temps}")
print(f"Promedio: {promedio:.2f}°C")
```

Enigma 8.3:

```
with open("mi_diario_mar.txt", "w", encoding="utf-8") as f:
    f.write("Observaciones del mar:\n")

observacion = "Vi un delfín saltando en la bahía"
with open("mi_diario_mar.txt", "a", encoding="utf-8") as f:
    f.write(f"- {observacion}\n")

with open("mi_diario_mar.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

Enigma 8.4:

```
contenido = """BITÁCORA DEL PUERTO DE ANCÓN
FECHA: 27-06-2026
ACTIVIDADES SOSPECHOSAS:
- Ruido sospechoso en sector norte
- Luces sospechosas en zona de boyas
- Embarcación no identificada
NOTA: Situación normal
"""

with open("bitacora_simple.txt", "w", encoding="utf-8") as f:
    f.write(contenido)

with open("bitacora_simple.txt", "r", encoding="utf-8") as f:
    texto = f.read()

conteo = texto.lower().count("sospechosa")
print(f"La palabra 'sospechosa' aparece {conteo} veces")
```

Capítulo 9

Enigma 9.1:

```
def conectar(ip, puerto):
    try:
        print(f"Conectando a {ip}:{puerto}...")
        if ip.startswith("192."):
            raise ConnectionError("Red local no disponible")
        print(" Conexión exitosa")
        return True
    except ConnectionError:
        print(" No se pudo conectar al generador")
        return False

conectar("192.168.1.1", 8080)
conectar("10.0.0.1", 8080)
```

Enigma 9.2:

```
archivo = input("Nombre del archivo: ")
try:
    with open(archivo, "r", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print(f"Archivo '{archivo}' no encontrado. Creando uno nuevo...")
    with open(archivo, "w", encoding="utf-8") as f:
        f.write("Datos marinos por defecto\n")
```

Enigma 9.3:

```
class OlaNoDetectadaError(Exception):
    pass

def detectar_ola(altura):
    if altura == 0:
        raise OlaNoDetectadaError("No se detectó ninguna ola")
    return f"Ola detectada: {altura}m"

print(detectar_ola(1.8))
try:
    detectar_ola(0)
except OlaNoDetectadaError as e:
    print(f"Error: {e}")
```

Enigma 9.4:

```
def conectar_generador():
    try:
        print("Conectando al generador submarino...")
        raise TimeoutError("Sin respuesta")
    except TimeoutError:
        print(" Error: tiempo de espera agotado")
    finally:
```



```

        print(" Cerrando conexión con el generador submarino")

conectar_generador()

```

Capítulo 10

Enigma 10.1:

```

# playas_peru.py
def lista_playas():
    return ["Ancón", "Tuquillo", "Hermosa", "Señoritas", "Grande"]

def mejor_playa():
    return "Ancón"

# Desde otro archivo: import playas_peru
# print(playas_peru.lista_playas())

```

Enigma 10.2:

```

if __name__ == "__main__":
    print("Módulo de playas - Modo autónomo")

```

Enigma 10.3:

```

from locuento_generador import desactivar
print(desactivar())

```

Capítulo 11

Enigma 11.1:

```

class TablaDeSurf:
    def __init__(self, marca, longitud, material):
        self.marca = marca
        self.longitud = longitud
        self.material = material

    def remar(self):
        print(f"Remando en {self.marca}")

t = TablaDeSurf("Channel Islands", 6.2, "epoxi")
t.remar()

```

Enigma 11.2:

```

class Ola:
    total_olas = 0
    def __init__(self, altura, frecuencia, direccion):
        self.altura = altura
        self.frecuencia = frecuencia
        self.direccion = direccion
        Ola.total_olas += 1

```

```
o1 = Ola(1.5, 10, "SO")
o2 = Ola(2.4, 15, "O")
print(f"Total olas creadas: {Ola.total_olas}")
```

Enigma 11.3:

```
class Playa:
    def __init__(self, nombre, ubicacion, tiene_muelle):
        self.nombre = nombre
        self.ubicacion = ubicacion
        self.tiene_muelle = tiene_muelle

    def info(self):
        m = "con muelle" if self.tiene_muelle else "sin muelle"
        return f"{self.nombre} ({self.ubicacion}) - {m}"

p1 = Playa("Ancón", "Lima", True)
p2 = Playa("Hermosa", "Lima", False)
p3 = Playa("Tuquillo", "Ancash", False)
print(p1.info())
print(p2.info())
print(p3.info())
```

Capítulo 12

Enigma 12.1:

```
class ZonaCostera:
    def __init__(self, nombre):
        self.nombre = nombre

class PlayaPublica(ZonaCostera):
    def __init__(self, nombre, tiene_acceso_libre):
        super().__init__(nombre)
        self.tiene_acceso_libre = tiene_acceso_libre

class PlayaPrivada(ZonaCostera):
    def __init__(self, nombre, dueno):
        super().__init__(nombre)
        self.dueno = dueno

pub = PlayaPublica("Ancón", True)
priv = PlayaPrivada("Costa Azul", "Soto")
print(f"Pública: {pub.nombre}, acceso libre: {pub.tiene_acceso_libre}")
print(f"Privada: {priv.nombre}, dueño: {priv.dueno}")
```

Enigma 12.2:

```
class SonidoMarino:
    def sonido(self):
        return "..."

class Ola(SonidoMarino):
```

```

def sonido(self):
    return "Swish!"

class Barco(SonidoMarino):
    def sonido(self):
        return "Buuu!"

for s in [Ola(), Barco()]:
    print(s.sonido())

```

Enigma 12.3:

```

class CuentaMaritima:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.__saldo = saldo

    def depositar(self, monto):
        self.__saldo += monto
        print(f"Depósito: +${monto}. Saldo: ${self.__saldo}")

    def retirar(self, monto):
        if monto <= self.__saldo:
            self.__saldo -= monto
            print(f"Retiro: -${monto}. Saldo: ${self.__saldo}")
            return True
        print("Saldo insuficiente")
        return False

c = CuentaMaritima("Eulogio", 500)
c.depositar(200)
c.retirar(100)
c.retirar(700)

```

Capítulo 13

Enigma 13.1:

```

def generar_reporte(olas, archivo="reporte_olas.txt"):
    with open(archivo, "w", encoding="utf-8") as f:
        f.write("=== REPORTE DE OLAS ===\n")
        f.write(f"Total: {len(olas)}\n")
        f.write(f"Surfeables: {len([o for o in olas if o.es_surfeable()])}\n")
        f.write(f"Artificiales: {len([o for o in olas if o.es_artificial()])}\n")
    print(f"Reporte generado: {archivo}")

```

Enigma 13.2:

```

class OlaTormenta(Ola):
    def __init__(self, altura, frecuencia, direccion, intensidad):
        super().__init__(altura, frecuencia, direccion)
        self.intensidad = intensidad

```

```
def info(self):  
    return f"TORMENTA: {self.altura}m, intensidad {self.intensidad}"
```

Enigma 13.3:

```
def mapa_calor(olas):  
    horas = {}  
    for o in olas:  
        h = o.timestamp[11:13]  
        horas[h] = horas.get(h, 0) + 1  
    for h in sorted(horas):  
        print(f"{h}: {'*' * horas[h]}")
```

Nota: Estas soluciones son una guía. No hay una única forma correcta de resolver los enigmas. Si tu solución funciona y tiene sentido, ¡es válida!

Sobre el Autor

Alex Goyzueta Delgado

Analista de datos, escritor y divulgador de tecnología. Peruano, nacido en Ancón, Lima, de padre y abuelos anconeros.

Alex no es surfista, pero creció viendo las olas desde el muelle de Ancón, escuchando las historias de los pescadores y observando cómo el mar marcaba el ritmo de vida del puerto. Hoy, desde su formación en análisis de datos, escribe historias que conectan la tecnología con los paisajes y la gente del Perú.

Después de *Código Asesino: El Misterio del Código Sagrado* (ambientado en Cusco), regresa con una historia costera que rinde homenaje a su tierra natal: Ancón, el puerto de su infancia y escenario de este misterio digital.

Contacto:

- Correo: alexgoyzueta2018@gmail.com
- Temas: Python, ciencia de datos, narrativa interactiva, tecnología y naturaleza
- Repositorio: github.com/SistGoyPeru/Libros

"El mar nunca deja de hablar. Solo necesitas saber escucharlo."